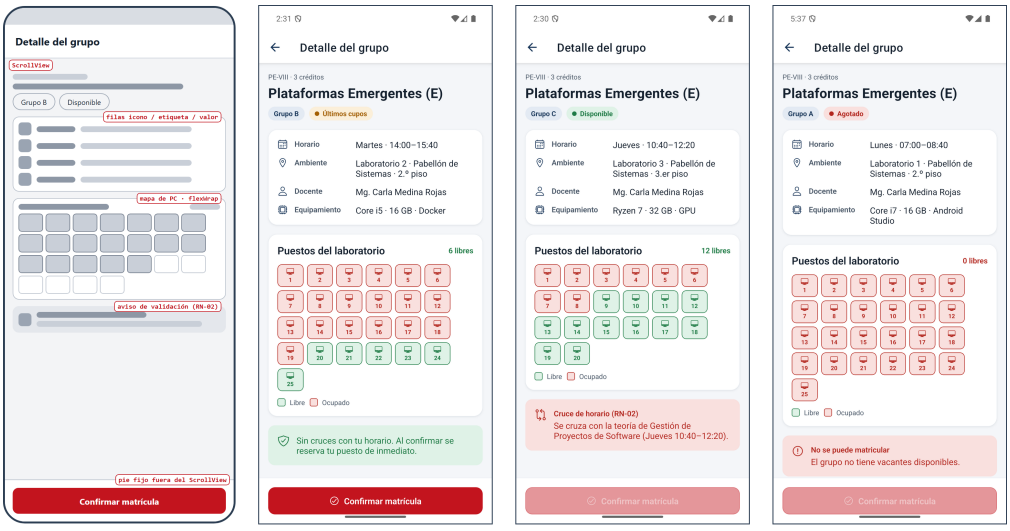
INFORME DE LABORATORIO

Plataformas Emergentes (E) – EPIS UNSA

I. INFORMACIÓN BÁSICA			
ASIGNATURA:	**PLATAFORMAS EMERGENTES (E)**		
TÍTULO DE PRÁCTICA:	Framework multiplataforma: interfaces de MatriLab EPIS con React Native		
NRO. PRÁCTICA:	04	**AÑO LECTIVO:**	2026-B
SEMESTRE ACADÉMICO:	VIII	**FECHA DE ENTREGA:**	20/10/2026
MODALIDAD / TIPO:	Grupal (3 integrantes)	**CURSO:**	LABORATORIO
ESTUDIANTE(S):	**1.** Lopez Arela, Ower Frank **2.** Zegarra Puma, Mauricio Eduardo **3.** Huamani Chañi, Diego Oswaldo		
DOCENTE:	M. Sc. Ing. R. Fabrizio Calienes Rodriguez		
REPOSITORIO:	`https://github.com/OwerLopez/matrilab-epis-react-native`		



MatriLab EPIS ejecutándose en Android: mockup, grupo disponible, bloqueo por cruce de horario y grupo agotado.

Índice General de Contenidos

1. Descripción general del programa **5**
1.1 Contexto y propósito . . . 5
1.2 Árbol de problemas . . . 5
1.3 Árbol de objetivos y alcance . . . 6
1.4 Actores y requisitos atendidos . . . 6
1.5 Elección del framework . . . 7

2. Diseño de interfaces **9**
2.1 Funcionalidades derivadas del árbol de objetivos . . . 9
2.2 Mapa de navegación . . . 9
2.3 Sistema visual . . . 9
2.4 Mockups de baja fidelidad . . . 10
2.5 Criterios de usabilidad y accesibilidad . . . 13

3. Componentes de React Native para la organización de la interfaz **14**
3.1 Cómo dibuja React Native una interfaz . . . 14
3.2 Flexbox en React Native . . . 14
3.3 Contenedores de organización . . . 16
3.4 Listas virtualizadas . . . 16
3.5 Posicionamiento, dimensiones y adaptación . . . 16
3.6 Componentes de interacción y retroalimentación . . . 17
3.7 Navegación nativa . . . 17

4. Descripción del proceso de desarrollo **18**
4.1 Entorno de trabajo . . . 18
4.2 Creación y ejecución del proyecto . . . 18
4.3 Arquitectura . . . 19
4.4 Reglas de negocio . . . 19
4.5 Estado global, tema y componentes propios . . . 21
4.6 Desarrollo de las interfaces . . . 21
4.7 Adaptación a la orientación y a otras plataformas . . . 32
4.8 Pruebas automatizadas y verificación . . . 33
4.9 Registro de cambios . . . 37
4.10 Repositorio y despliegue . . . 38

5. Apreciación de la herramienta **40**
5.1 Características favorables . . . 40
5.2 Características desfavorables . . . 40

6. Conclusiones **41**

7. Anexos **42**
Anexo A. Tema oscuro . . . 42
Anexo B. Desarrollo con Expo Go . . . 42

8. Referencias **43**

Índice de figuras

1. Árbol de problemas del proceso de matrícula de laboratorios de la EPIS-UNSA 5
2. Árbol de objetivos: los medios fundamentales se traducen en interfaces de MatriLab 6
3. Mapa de navegación de MatriLab: pila nativa raíz y pestañas por rol 10
4. Mockups de baja fidelidad de las interfaces del estudiante (I1 a I6) 11
5. Mockups de baja fidelidad de las interfaces del docente y de administración (I7 a I12) 12
6. Patrones de layout con Flexbox utilizados en MatriLab 15
7. Servidor Metro iniciado con npx expo start: QR para Expo Go y menú de atajos 18
8. Expo Go en el emulador: (a) servidor de desarrollo MatriLab EPIS detectado en la red local, (b) menú de desarrollo 19
9. Arquitectura por capas de MatriLab EPIS 20
10. Inicio de sesión: (a) mockup, (b) pantalla inicial, (c) validación del dominio institucional, (d) formulario con el teclado abierto y la contraseña visible 21
11. Panel del estudiante: (a) mockup, (b) tema claro con una matrícula, (c) tema oscuro después de matricularse en Plataformas Emergentes 22
12. Panel del estudiante: (a) dos laboratorios matriculados, (b) diálogo nativo para anular una matrícula 23
13. Catálogo: (a) mockup, (b) todos los grupos, (c) filtro Sin cruce de horario 23
14. Catálogo: (a) búsqueda por nombre de curso, (b) estado vacío sin resultados, (c) filtro Con vacantes 24
15. Detalle del grupo: (a) mockup, (b) grupo B disponible, (c) grupo C bloqueado por cruce con la teoría (RN-02), (d) grupo A agotado 25
16. Pase de laboratorio: (a) mockup, (b) pase emitido tras confirmar la matrícula, (c) hoja nativa de compartir 26
17. Horario: (a) mockup, (b) vista semanal, (c) vista de lista por día, (d) semana después de matricularse en Plataformas Emergentes 27
18. Panel del docente: (a) mockup, (b) panel con los grupos a cargo, (c) grupos después de registrar una asistencia 27
19. Nómina del grupo: (a) mockup, (b) dos asistencias registradas con su hora 28
20. Nómina: (a) búsqueda por apellido, (b) exportación de la nómina en CSV con la hoja nativa de compartir 29
21. Escáner: (a) mockup, (b) cámara activa en el emulador, (c) pase validado, (d) código rechazado 30
22. Gestión de laboratorios: (a) mockup, (b) lista de ambientes, (c) edición en la hoja modal, (d) laboratorio inhabilitado y capacidad ajustada 31
23. Configuración de la matrícula: (a) mockup, (b) ventana abierta y reglas 31
24. Configuración: (a) fecha inválida rechazada, (b) confirmación nativa al guardar, (c) grupos agotados al final del formulario 32
25. Perfil y ajustes: (a) mockup, (b) perfil del estudiante, (c) perfil del docente 33
26. Orientación horizontal en el emulador: (a) laboratorios en dos columnas, (b) horario semanal . 34
27. La misma aplicación en un navegador web: inicio de sesión, panel del estudiante y catálogo . . 35
28. Versión web en distintos anchos: (a) 1366 px con tres columnas, (b) 820 px con dos columnas, (c) inicio de sesión centrado 36
29. Verificación de tipos sin errores y 31 pruebas aprobadas en la consola 37
30. Repositorio del proyecto en GitHub 39
31. Flujo de GitHub Actions: trabajos construir y desplegar completados 39
32. Tema oscuro: (a) panel del estudiante, (b) catálogo, (c) detalle con la regla de un grupo por curso, (d) horario 42

Índice de tablas

1. Actores del sistema y lo que cada uno puede hacer en la aplicación 7
2. Requisitos funcionales del LAB 01 y cómo se resuelven en la aplicación 7
3. Trazabilidad entre medios del árbol de objetivos, funcionalidades e interfaces 9
4. Tokens de diseño de la aplicación 10
5. Componentes del núcleo y la vista nativa que los representa en cada plataforma (según [2]) . . 14
6. Propiedades de layout más usadas y dónde se aplican 15
7. Configuración de las listas virtualizadas de la aplicación 16
8. Componentes de interacción utilizados 17
9. Entorno de desarrollo y ejecución 18
10. Reglas de negocio aplicadas al confirmar una matrícula 20
11. Pruebas unitarias agrupadas por regla verificada 37
12. Cambios realizados durante el desarrollo y su motivo 38

1. DESCRIPCIÓN GENERAL DEL PROGRAMA

1.1 Contexto y propósito

MatriLab EPIS es una aplicación móvil para gestionar la matrícula en los grupos de laboratorio de la Escuela Profesional de Ingeniería de Sistemas de la UNSA. Es el mismo dominio que el equipo trabajó en el LAB 01 (selección de herramienta y requisitos) y en el LAB 02 (prototipo RMAD en FlutterFlow). En esta práctica la aplicación se reconstruye con un **framework multiplataforma**, **React Native**, usando únicamente los componentes propios de la plataforma para organizar cada interfaz.

El programa atiende a tres usuarios. El **estudiante** consulta la oferta de grupos con un semáforo de vacantes, se matricula sin cruces de horario y obtiene un pase digital con código QR. El **docente o jefe de prácticas** revisa la nómina de sus grupos y registra la asistencia leyendo el QR con la cámara del teléfono. La **coordinación de laboratorios** define la capacidad de cada ambiente y la ventana de matrícula. Las doce pantallas comparten un mismo código que se ejecuta como aplicación nativa en Android y como aplicación web.

Los datos de esta versión son de demostración y viven en memoria: el objetivo de la práctica es el diseño y la organización de las interfaces. Las reglas de negocio (cruce de horarios, aforo, ventana de matrícula y verificación del pase) sí son reales y se verifican con pruebas automatizadas.

1.2 Árbol de problemas

El diseño parte del diagnóstico del LAB 01 sobre el proceso de matrícula de laboratorios. Se aplicó la técnica del árbol de problemas de la metodología del marco lógico [17]: un problema central, sus causas directas e indirectas y los efectos que produce (Figura 1).

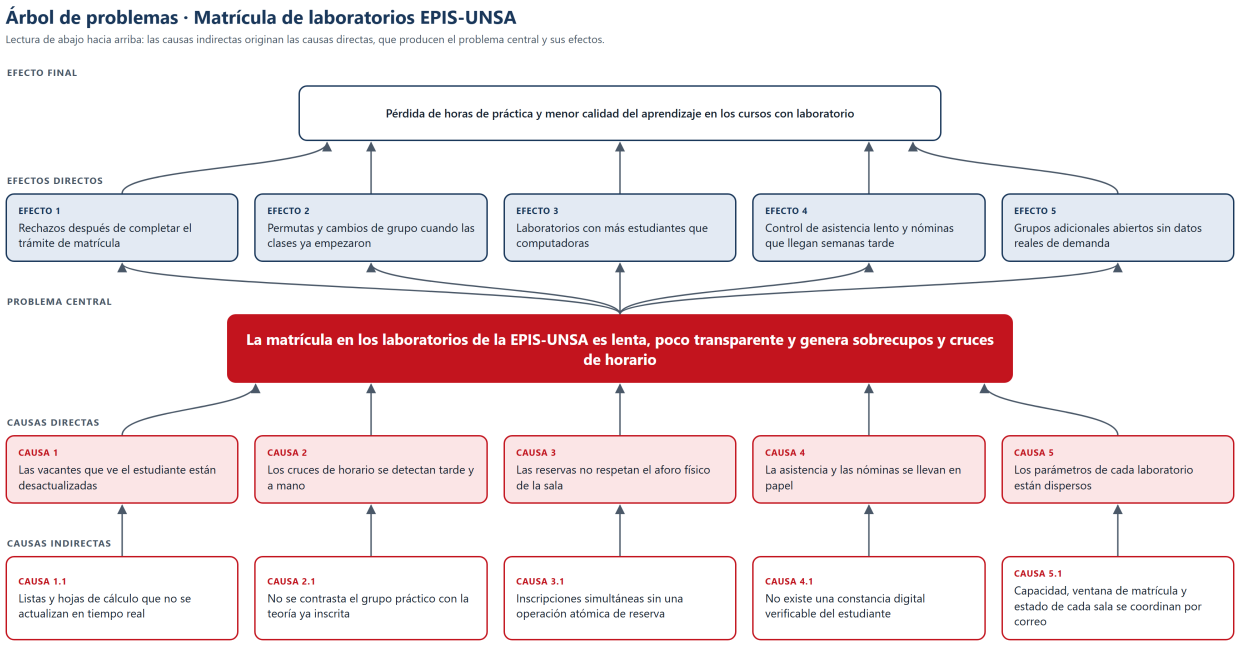


Figura 1. Árbol de problemas del proceso de matrícula de laboratorios de la EPIS-UNSA

El problema central es que **la matrícula en los laboratorios es lenta, poco transparente y genera sobrecupos y cruces de horario**. Las causas de fondo son listas que no se actualizan en tiempo real, la falta de contraste entre el grupo práctico y la teoría inscrita, inscripciones simultáneas sin una operación atómica, la ausencia de una constancia digital y la coordinación de las salas por correo. Los efectos llegan al aula: rechazos al final del trámite, permutas cuando las clases ya empezaron, salas con más estudiantes que computadoras y nóminas tardías.

1.3 Árbol de objetivos y alcance

Cada condición negativa se convirtió en la situación deseada (Figura 2). Los **medios fundamentales**, en la base del árbol, son acciones concretas que el software puede ejecutar; por eso se tomaron como origen de las funcionalidades. Debajo de cada medio se indican las interfaces de MatriLab que lo implementan.

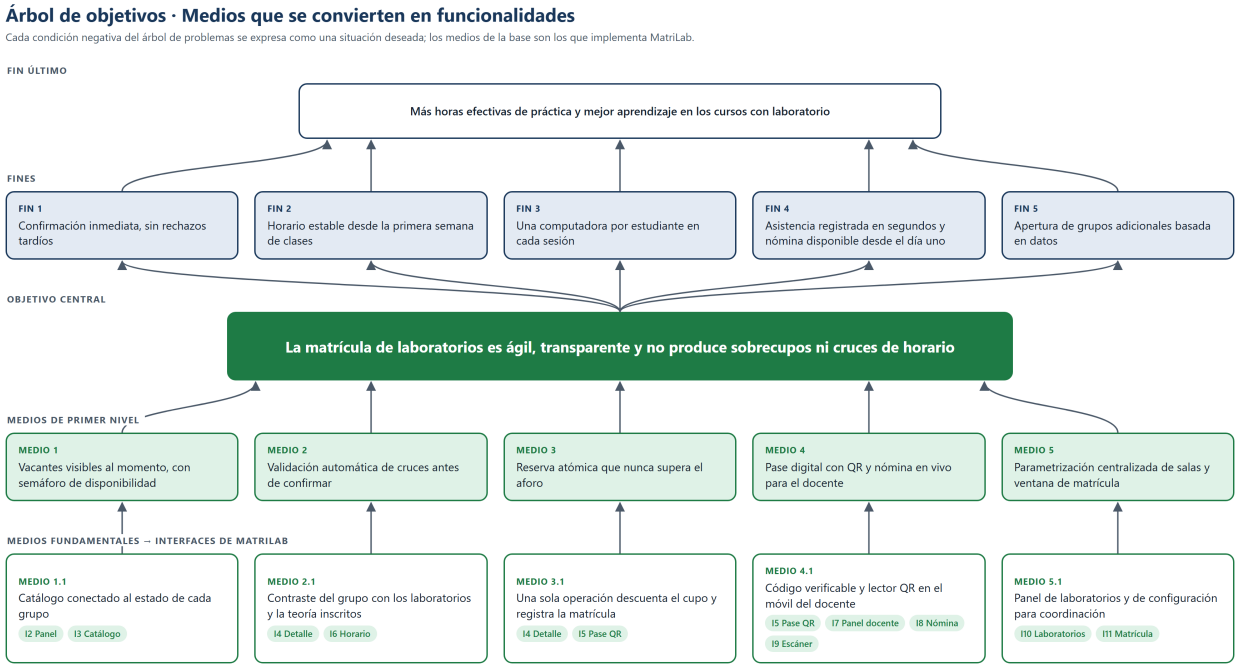


Figura 2. Árbol de objetivos: los medios fundamentales se traducen en interfaces de MatriLab

El alcance de la práctica cubre los cinco medios con **doce interfaces** (I1 a I12). La autenticación real con Google Workspace y la persistencia en la nube, que en el LAB 02 se resolvieron con Firebase, se simulan aquí con un inicio de sesión de demostración y un estado local, de modo que cualquier evaluador puede probar la aplicación sin cuentas ni conexión.

1.4 Actores y requisitos atendidos

Tabla 1. Actores del sistema y lo que cada uno puede hacer en la aplicación

Rol	Cuenta de demostración	Responsabilidades en MatriLab	Interfaces
Estudiante	Lopez Arela, Ower Frank · CUI 20213412	Consultar la oferta, matricularse sin cruces, ver su horario, mostrar su pase QR y anular una matrícula dentro de la ventana.	I1–I6, I12
Docente / jefe de prácticas	Mg. Carla Medina Rojas (ficticia)	Revisar la ocupación de sus grupos, consultar la nómina, registrar asistencia con QR o código manual y exportar la nómina.	I1, I7–I9, I12
Coordinación de laboratorios	`laboratorios.epis@unsa.edu.pe`	Ajustar capacidad y estado de cada ambiente, definir la ventana de matrícula y revisar los grupos agotados.	I1, I10–I12

Tabla 2. Requisitos funcionales del LAB 01 y cómo se resuelven en la aplicación

Código	Requisito	Cómo se resuelve	Interfaz
RF-01	Autenticación institucional	Validación del dominio @unsa.edu.pe y de la contraseña; acceso de demostración por rol.	I1
RF-02	Catálogo de cursos y grupos	Lista agrupada por curso con buscador y filtros.	I3
RF-03	Vacantes en vivo	Semáforo y barra de ocupación que se recalculan al matricular o anular.	I2, I3, I4
RF-04	Validación anti-cruces	Contraste del grupo con los laboratorios y la teoría inscritos (RN-02).	I4
RF-05	Reserva atómica	El cupo se descuenta y la matrícula se registra en una sola operación.	I4
RF-06	Ficha digital con QR	Pase con código QR firmado y código de verificación de 8 caracteres.	I5
RF-07	Horario consolidado y nómina	Cuadrícula semanal con teoría y laboratorios; nómina por grupo.	I6, I7, I8
RF-08	Asistencia por escaneo QR	Cámara del teléfono con lector QR o ingreso manual del código.	I9
RF-09	Parametrización de laboratorios	Capacidad, estado y datos de cada sala; ventana de matrícula validada.	I10, I11
RF-10	Exportación de nóminas	CSV con separador punto y coma compartido con la hoja nativa del sistema.	I8

1.5 Elección del framework

La guía permite Flutter, React Native o Xamarin. El equipo eligió **React Native 0.88 con Expo SDK 58** por tres razones. Primero, React Native dibuja la interfaz con **vistas nativas** de cada sistema operativo: un `View` es un `ViewGroup` en Android y un `UIView` en iOS [2], lo que responde directamente al requisito de usar los componentes propios de la plataforma. Segundo, el LAB 02 ya se había construido sobre Flutter mediante FlutterFlow, y usar otro framework permite comparar ambos enfoques con evidencia propia. Tercero, Expo [14] permite ejecutar la aplicación en un emulador o teléfono con Expo Go sin compilar proyectos nativos y exportarla a la web con el mismo código. Se complementó con TypeScript, React Navigation 7 para la navegación nativa, `expo-camera` para el lector QR y Jest para las pruebas.

2. DISEÑO DE INTERFACES

2.1 Funcionalidades derivadas del árbol de objetivos

Cada medio fundamental del árbol de objetivos se descompuso en funcionalidades concretas (Tabla 3). Ninguna pantalla se agregó sin un medio que la justifique.

Tabla 3. Trazabilidad entre medios del árbol de objetivos, funcionalidades e interfaces

Medio	Funcionalidad	Interfaz
1.1 Catálogo conectado	F-01 Iniciar sesión con correo institucional y elegir rol	I1 Inicio de sesión
1.1 Catálogo conectado	F-02 Ver resumen personal: ventana, créditos, vacantes y laboratorios matriculados	I2 Panel del estudiante
1.1 Catálogo conectado	F-03 Explorar grupos por curso con semáforo, buscador y filtros	I3 Catálogo
2.1 Contraste de horario	F-04 Revisar el detalle del grupo y el mapa de puestos; validar cruces antes de confirmar	I4 Detalle del grupo
2.1 Contraste de horario	F-05 Consultar el horario semanal con teoría y laboratorios	I6 Horario
3.1 Operación única	F-06 Confirmar la matrícula con reserva atómica del cupo	I4 Detalle del grupo
4.1 Código verificable	F-07 Mostrar y compartir el pase con QR y código de verificación	I5 Pase QR
4.1 Código verificable	F-08 Monitorear la ocupación de los grupos del docente	I7 Panel docente
4.1 Código verificable	F-09 Consultar la nómina, marcar asistencia y exportar CSV	I8 Nómina
4.1 Código verificable	F-10 Leer el QR con la cámara o digitar el código	I9 Escáner
5.1 Parametrización	F-11 Ajustar capacidad, estado y datos de cada laboratorio	I10 Laboratorios
5.1 Parametrización	F-12 Configurar la ventana de matrícula y las reglas; revisar grupos agotados	I11 Matrícula
Transversal	F-13 Elegir tema claro u oscuro, ajustar avisos y cerrar sesión	I12 Perfil

2.2 Mapa de navegación

La estructura de navegación se definió antes de dibujar las pantallas (Figura 3). Una **pila nativa** raíz contiene el inicio de sesión, tres navegadores de **pestañas inferiores** (uno por rol) y tres pantallas que se apilan encima: el detalle del grupo, que se presenta como hoja modal, el pase QR y la nómina. Cada rol ve solo sus opciones y el botón Atrás del sistema se comporta como el usuario espera.

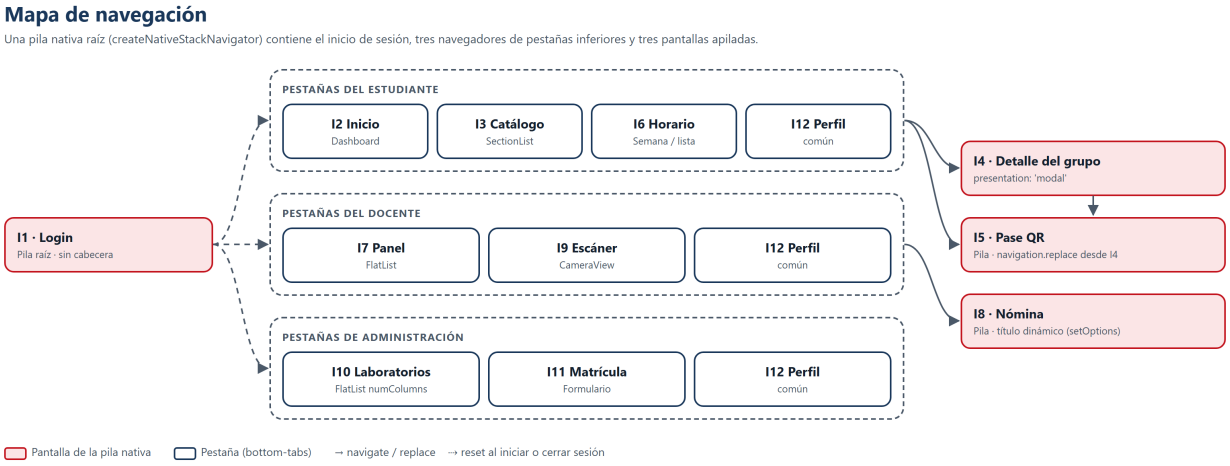


Figura 3. Mapa de navegación de MatriLab: pila nativa raíz y pestañas por rol

2.3 Sistema visual

Para que las doce interfaces sean consistentes se definieron colores, espacios y tamaños de texto comunes. Se conservaron los colores institucionales usados desde el LAB 01 y se definió una variante oscura de cada color, porque la aplicación sigue el tema del sistema operativo.

Tabla 4. Tokens de diseño de la aplicación

Token	Claro	Oscuro	Uso
primario	#19375A	#7FA7D9	Botones principales, cabeceras de tarjetas, grupo seleccionado.
marca	#C3141E	#FF6B6B	Acciones decisivas (Ingresar, Confirmar matrícula) y pestaña activa.
éxito / alerta / peligro	#1E7B45 / #A35F00 / #B3261E	variantes claras	Semáforo de vacantes: verde, ámbar y rojo.
fondo / superficie	#F4F6F9 / #FFFFFF	#0F1720 / #17212C	Fondo de pantalla y de tarjetas.
espacio y radio	4 · 8 · 12 · 16 · 24 · 32 / 8 · 12 · 16	igual	Márgenes, rellenos, separación entre elementos y esquinas.

2.4 Mockups de baja fidelidad

Con las funcionalidades y la navegación definidas se dibujaron mockups de baja fidelidad de las doce interfaces (Figuras 4 y 5). Se usaron solo grises, sin textos reales, para discutir la **organización** de cada pantalla antes que su apariencia. Cada región lleva una etiqueta roja con el componente de React Native previsto y los recuadros discontinuos marcan contenedores de layout. En la sección 4 cada mockup aparece junto a la captura real de la pantalla.

Las decisiones de organización que quedaron fijadas en los mockups fueron:

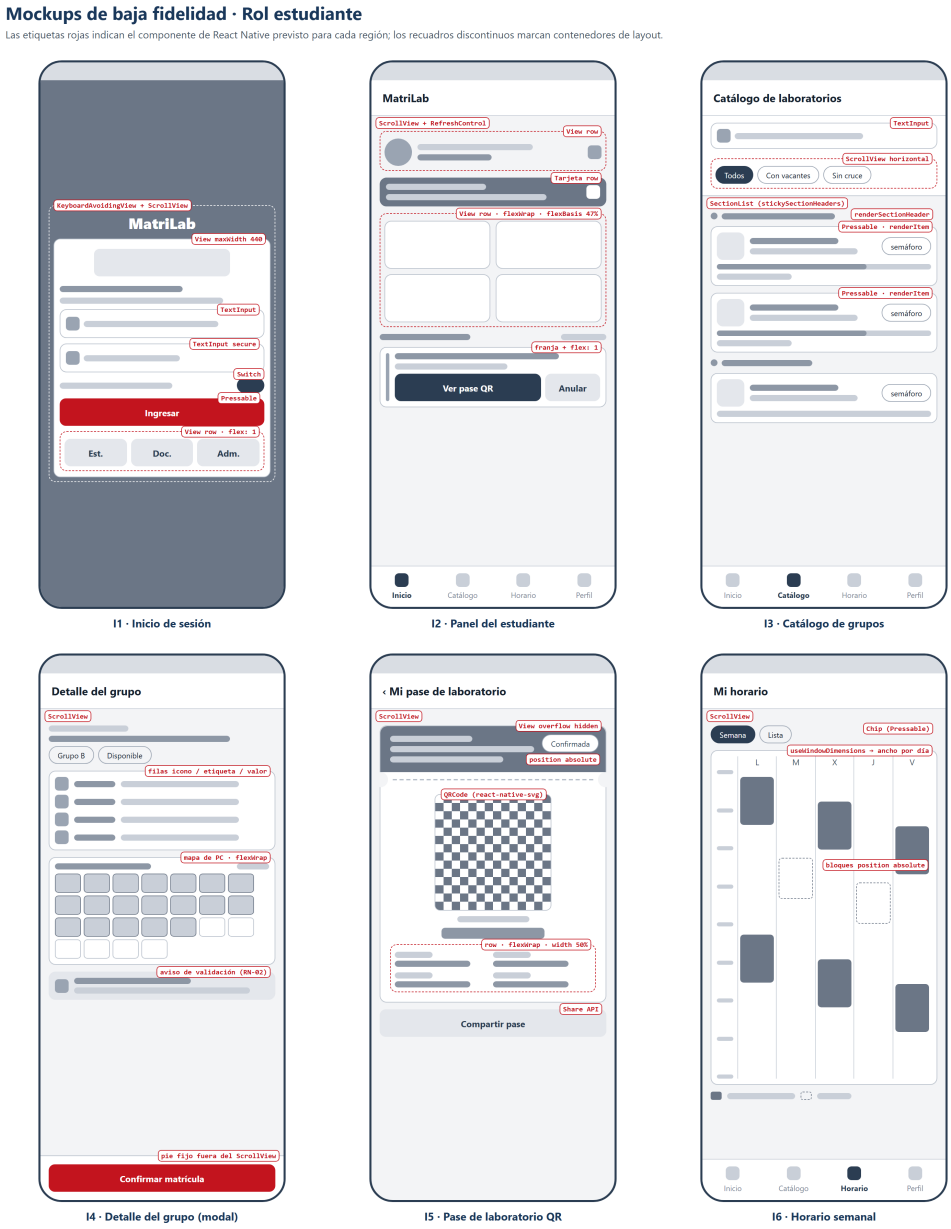


Figura 4. Mockups de baja fidelidad de las interfaces del estudiante (I1 a I6)

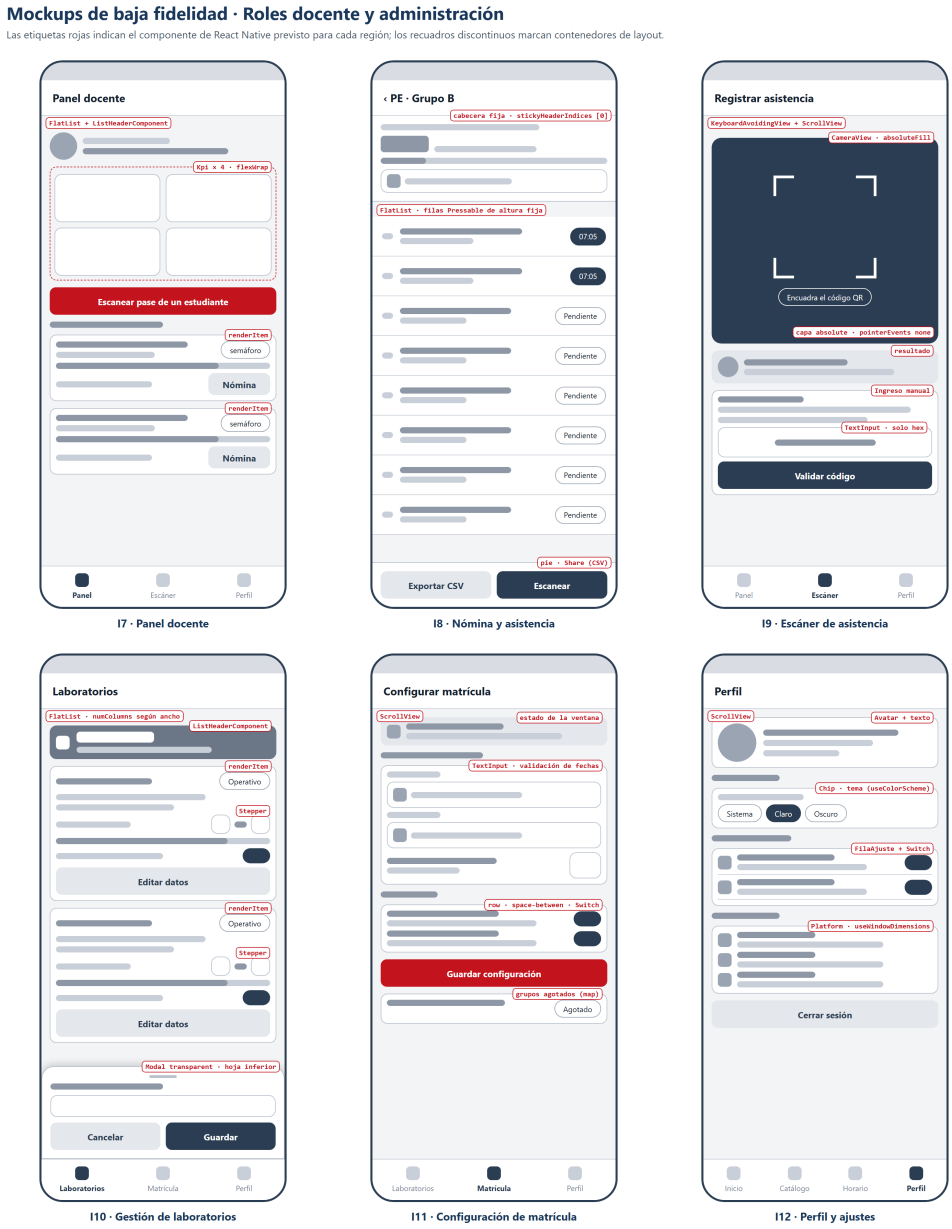


Figura 5. Mockups de baja fidelidad de las interfaces del docente y de administración (I7 a I12)

- **Listas largas virtualizadas.** El catálogo, el panel docente, la nómina y los laboratorios se planificaron como listas que solo dibujan lo visible, no como un desplazamiento simple con muchos elementos.
- **Acción principal fija.** En el detalle del grupo y en la nómina el botón principal queda en un pie fuera del área desplazable, siempre a la vista.
- **Superposiciones.** El sello del pase, la guía de encuadre del escáner y los bloques del horario se colocan con posición absoluta sobre su contenedor.
- **Cuadrículas que se reacomodan.** Los indicadores y el mapa de puestos saltan de línea solos, de modo que el número de columnas depende del ancho disponible.

2.5 Criterios de usabilidad y accesibilidad

- Área táctil mínima de **48 dp** en botones y campos.
- Cada control tiene una etiqueta de accesibilidad; las filas de la nómina se anuncian como casillas con su estado.
- Los mensajes de error y validación se leen automáticamente al aparecer.
- El semáforo nunca depende solo del color: siempre lleva el texto *Disponible*, *Últimos cupos* o *Agotado*.
- Tema claro y oscuro con contraste suficiente; se respeta la escala de texto del sistema.

3. COMPONENTES DE REACT NATIVE PARA LA ORGANIZACIÓN DE LA INTERFAZ

3.1 Cómo dibuja React Native una interfaz

En React Native la interfaz se describe con componentes de React, pero lo que aparece en pantalla no es HTML: el renderizador **Fabric** traduce cada componente del núcleo a una vista nativa del sistema operativo [2], [13]. Las posiciones y tamaños se calculan con **Yoga**, un motor de layout multiplataforma que implementa Flexbox [15]. La Tabla 5 resume la correspondencia documentada para los componentes básicos.

Tabla 5. Componentes del núcleo y la vista nativa que los representa en cada plataforma (según [2])

Componente	Android	iOS	Web	Función
View	ViewGroup	UIView	div	Contenedor con flexbox, estilo, toques y accesibilidad.
Text	TextView	UITextView	p	Muestra y anida texto con estilos.
Image	ImageView	UIImageView	img	Muestra imágenes locales o remotas.
ScrollView	ScrollView	UIScrollView	div desplazable	Contenedor genérico con desplazamiento.
TextInput	EditText	UITextField	input	Entrada de texto con el teclado del sistema.

Por esta correspondencia la aplicación se siente propia del sistema: el interruptor de la configuración es el de Android, el teclado es el del teléfono, los diálogos y la hoja de compartir son los del sistema y la pila de navegación usa las transiciones nativas (se ven en las capturas de la sección 4).

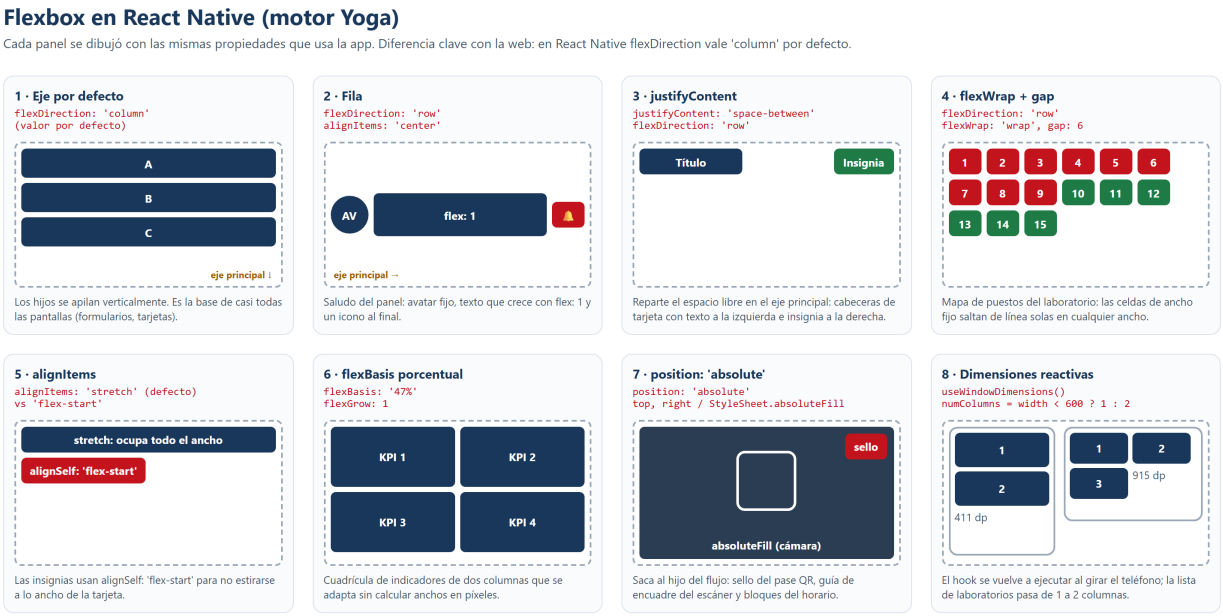


Figura 6. Patrones de layout con Flexbox utilizados en MatriLab

3.2 Flexbox en React Native

Toda la organización espacial se hace con Flexbox. Un contenedor define un **eje principal** (`flexDirection`) y distribuye a sus hijos a lo largo de él (`justifyContent`) y del eje transversal (`alignItems`). La documentación oficial advierte que, aunque el modelo es el de CSS, **los valores por defecto cambian** [1]: `flexDirection` vale `column` (en la web vale `row`), `alignContent` vale `flex-start`, `flexShrink` vale 0 y `flex` acepta un solo número. La Figura 6 reproduce los ocho patrones de layout que más se repiten en MatriLab.

Tabla 6. Propiedades de layout más usadas y dónde se aplican

Propiedad	Valores usados	Dónde se ve en MatriLab
flexDirection	row, column	Filas icono/texto/acción; formularios en columna.
flex	1	El texto de una fila ocupa el espacio que dejan el avatar y la insignia.
justifyContent	space-between, center	Cabecera de tarjeta con título e insignia; botones centrados.
alignItems / alignSelf	center, flex-start, stretch	Insignias que no se estiran; columna de datos del pase.
flexWrap + gap	wrap, 6–12 dp	Mapa de puestos del laboratorio y KPI en dos columnas.
flexBasis / flexGrow	47 %, 1	Cada KPI ocupa media fila y absorbe el sobrante.
position	absolute con top/left/right/bottom	Sello del pase, guía del escáner, bloques del horario.
width en porcentaje	50 % y porcentajes calculados	Datos del pase en dos columnas; relleno de la barra de ocupación.

3.3 Contenedores de organización

- **View.** Es el bloque de construcción de toda pantalla. MatriLab no usa ninguna librería de componentes visuales: tarjetas, botones, chips, insignias y barras de progreso se construyeron sobre View, Text y Pressable.
- **SafeAreaView.** Evita que el contenido quede bajo la barra de estado o la muesca. La documentación marca el del núcleo como obsoleto y recomienda `react-native-safe-area-context` [16], que es el que usa la aplicación.
- **ScrollView.** Contenedor desplazable que monta todos sus hijos a la vez [3]. Se usa en pantallas de contenido acotado (panel del estudiante, detalle, pase, horario, configuración, perfil) y en forma horizontal para la fila de filtros del catálogo.
- **KeyboardAvoidingView.** Ajusta el espacio cuando aparece el teclado [8]; su comportamiento difiere entre Android e iOS, por eso se configura según la plataforma (ver Figura 10, panel d).
- **Modal.** Presenta contenido sobre la pantalla actual [9]. Con fondo transparente se construyó una hoja inferior para editar los datos de un laboratorio (Figura 22, panel c).

3.4 Listas virtualizadas

Cuando la cantidad de elementos crece, un ScrollView desperdicia memoria porque dibuja todo aunque no se vea. **FlatList** y **SectionList** solo montan los elementos cercanos a la zona visible [4], [5], [18]. La Tabla 7 resume cómo se configuró cada lista.

Tabla 7. Configuración de las listas virtualizadas de la aplicación

Interfaz	Componente	Configuración	Motivo
I3 Catálogo	SectionList	Una sección por curso con encabezado fijo al desplazarse y mensaje para lista vacía.	Agrupar los grupos por curso sin perder de vista a cuál pertenecen.
I7 Panel docente	FlatList	Saludo e indicadores como cabecera de la propia lista.	Desplazar todo junto sin anidar listas, lo que anularía la virtualización.
I8 Nómina	FlatList	Resumen fijo arriba (sticky header) y filas de alto constante.	Asistencia y buscador siempre visibles sobre la lista de estudiantes.
I10 Laboratorios	FlatList	Número de columnas calculado según el ancho de la ventana.	Rejilla de 1, 2 o 3 columnas en teléfono, horizontal o pantalla ancha.

> FlatList no permite cambiar el número de columnas mientras está montada. Por eso la lista de laboratorios se vuelve a crear cuando cambia el ancho; el resultado se ve en las Figuras 26 y 28.

3.5 Posicionamiento, dimensiones y adaptación

- **Posición absoluta.** Saca a un elemento del flujo y lo ubica respecto de su contenedor; `StyleSheet.absoluteFill` lo cubre por completo, como la vista de cámara del escáner.
- **Porcentajes.** Ancho y alto aceptan porcentajes del contenedor [12]; así se dibuja el relleno de la barra de ocupación.
- **useWindowDimensions** devuelve el ancho y alto de la ventana y se actualiza al rotar [10]. Con él se calculan el ancho de cada día del horario, el tamaño del QR y las columnas de la lista de laboratorios.
- **Platform** permite pequeñas diferencias por sistema [11]; **StyleSheet** agrupa los estilos y ofrece la línea más fina que puede dibujar la pantalla, usada en bordes y separadores [6].
- **useColorScheme** informa si el sistema está en modo claro u oscuro y define la paleta de toda la aplicación.

3.6 Componentes de interacción y retroalimentación

Tabla 8. Componentes de interacción utilizados

Componente o API	Uso en MatriLab
Pressable	Base de botones, chips, tarjetas tocables y filas de la nómina, con efecto al presionar y ripple de Android [7].
TextInput	Correo y contraseña oculta, buscadores, fechas de la ventana y código de verificación (solo hexadecimal).
Switch	Mantener sesión, habilitar laboratorio, reglas de matrícula y avisos del perfil.
ActivityIndicator / RefreshControl	Indicador al confirmar la reserva y gesto de deslizar para actualizar el panel.
Alert	Diálogo nativo para confirmar la anulación de una matrícula y avisar cambios guardados.
Share	Hoja nativa de compartir para el pase y para la nómina en CSV.
CameraView (expo-camera)	Vista previa de la cámara con lectura de códigos QR [19].

3.7 Navegación nativa

React Native no incluye un navegador en su núcleo; se usó **React Navigation** [20], [21]. La pila nativa delega en los componentes del sistema (fragmentos en Android y `UINavigationController` en iOS), por lo que las transiciones y el gesto de volver son los del sistema. El detalle del grupo se presenta como hoja modal y cada rol tiene su barra de pestañas inferior, cuyos iconos cambian entre la versión rellena y la de contorno según la pestaña activa.

4. DESCRIPCIÓN DEL PROCESO DE DESARROLLO

4.1 Entorno de trabajo

Tabla 9. Entorno de desarrollo y ejecución

Elemento	Detalle
Sistema operativo y editor	Windows 11 Pro · Visual Studio Code
Node.js / npm	22.14.0 / 10.9.2
Framework	React Native 0.88 · React 19.3 · Expo SDK 58 (CLI 58.1.13) · TypeScript 6.0
Emulador	Android Studio · Pixel 7a · Android 16 (API 36) · 1080 × 2400 px, 420 dpi (411 × 914 dp)
Cliente de ejecución	Expo Go 58.0.5 instalado en el emulador
Control de versiones y despliegue	Git, GitHub y GitHub Actions con GitHub Pages

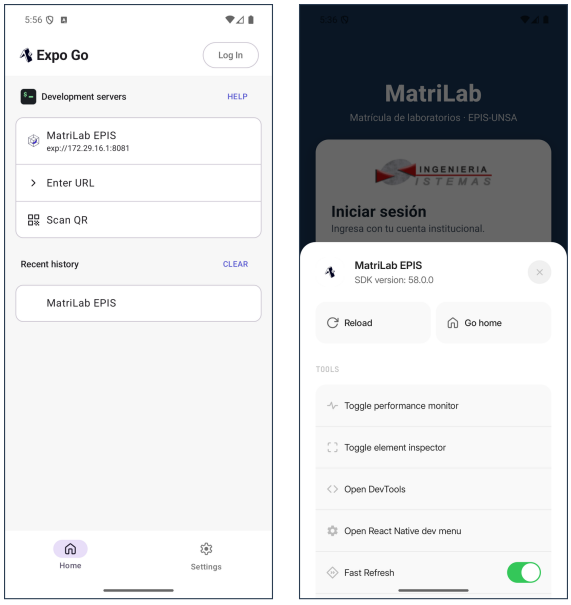
4.2 Creación y ejecución del proyecto

El proyecto se creó con la plantilla en blanco de TypeScript de Expo y las dependencias se instalaron con el instalador de Expo, que elige versiones compatibles con el SDK. Para ejecutar la aplicación se inicia el servidor **Metro**, que empaqueta el código y lo sirve al dispositivo. La Figura 7 muestra la consola de Metro con el código QR para abrir el proyecto en Expo Go y los atajos de teclado: *a* abre el emulador Android, *w* la versión web y *r* recarga la aplicación.



Figura 7. Servidor Metro iniciado con npx expo start: QR para Expo Go y menú de atajos

En el emulador, Expo Go detecta el servidor de desarrollo en la red local y muestra el proyecto *MatriLab EPIS* listo para abrirse; al tocarlo descarga el paquete desde Metro y ejecuta la aplicación. Cada cambio guardado en el editor se refleja en uno o dos segundos gracias a *Fast Refresh*, sin perder el estado de la pantalla. Expo Go ofrece además un menú de desarrollo con recarga, inspector de elementos, monitor de rendimiento y herramientas de depuración (Figura 8).



(a) Servidor detectado (b) Menú de desarrollo

Figura 8. Expo Go en el emulador: (a) servidor de desarrollo MatriLab EPIS detectado en la red local, (b) menú de desarrollo

El código se organizó por responsabilidad: un **dominio** con las reglas de negocio sin dependencias de React, un **estado** global, un **tema** con las paletas clara y oscura, **componentes** visuales reutilizables, la **navegación** y una carpeta de **pantallas** por rol.

4.3 Arquitectura

La Figura 9 muestra las capas. La presentación solo conoce el estado a través de hooks; el estado aplica las reglas del dominio, y el dominio está formado por funciones puras que no dependen de React ni de la plataforma. Esa separación permitió probar las reglas de negocio en un segundo, sin emulador.

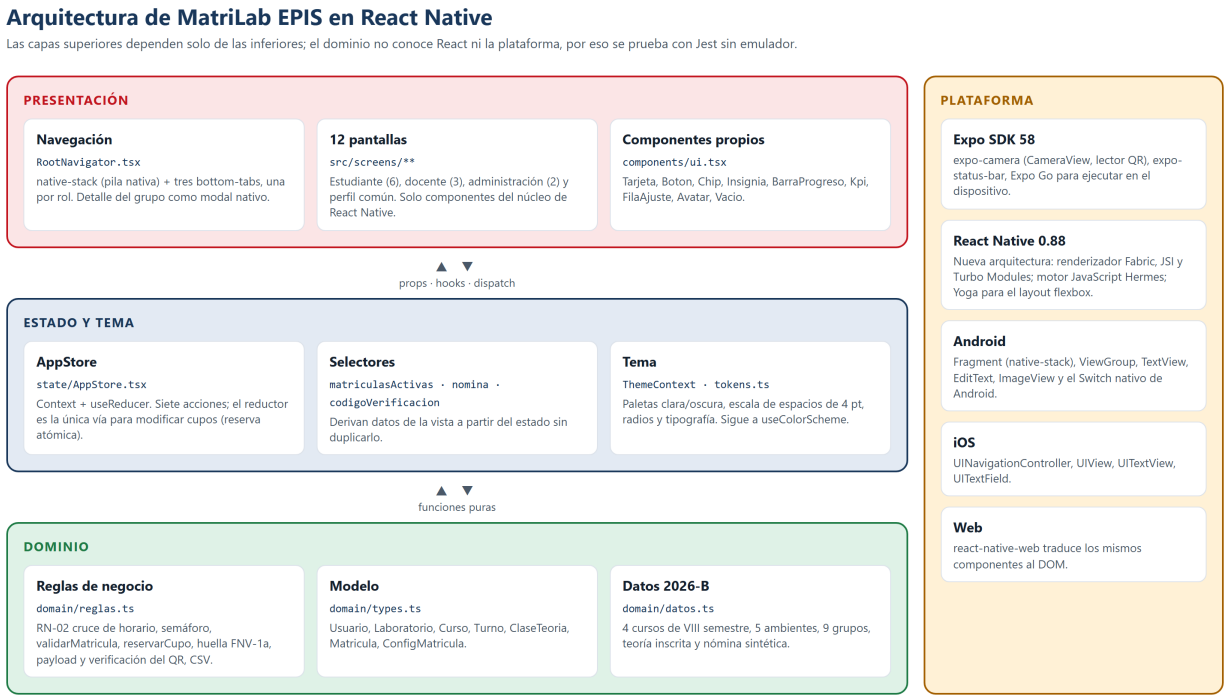


Figura 9. Arquitectura por capas de MatriLab EPIS

4.4 Reglas de negocio

Los horarios se representan en minutos desde la medianoche, lo que reduce la regla de cruce a comparar intervalos. La Tabla 10 resume las reglas que la aplicación aplica antes de confirmar una matrícula, en el mismo orden del proceso BPMN del LAB 01. Cada una produce un mensaje distinto en la pantalla del detalle del grupo.

Tabla 10. Reglas de negocio aplicadas al confirmar una matrícula

Orden	Regla	Comportamiento	Se ve en
1	Ventana de matrícula	Fuera de la fecha de apertura y cierre no se puede confirmar.	I2, I11
2	Grupo ya matriculado	No se puede tomar dos veces el mismo grupo.	I4
3	Un grupo por curso	Rechaza un segundo grupo del mismo curso.	Figura 32 (c)
4	Límite de laboratorios	Máximo configurable por estudiante (4 por defecto).	I2, I11
5	RN-02 cruce de horario	Rechaza el grupo si se solapa con otro laboratorio o con la teoría inscrita; los extremos no cuentan.	Figura 15 (c)
6	Cupo disponible	Sin vacantes el botón de confirmación queda deshabilitado.	Figura 15 (d)
7	Reserva atómica	El cupo se descuenta y la matrícula se registra en una sola operación; nunca se supera el aforo.	I2, I3
8	Pase verificable	El QR lleva una huella de los datos; si se altera, el escáner lo rechaza.	Figura 21 (d)

4.5 Estado global, tema y componentes propios

El estado de la aplicación (usuario, grupos, matrículas, laboratorios, configuración y asistencias) se mantiene en un único almacén construido con el contexto y un reductor de React. El reductor es la única vía para cambiar los cupos, de modo que dos toques seguidos en *Confirmar matrícula* no ocupan dos puestos. El tema sigue al sistema operativo, aunque el usuario puede fijar el modo claro u oscuro desde su perfil; al cambiarlo se repinta toda la aplicación (Anexo A). Los componentes visuales propios (tarjeta, botón, chip, insignia, barra de progreso, indicador, fila de ajuste, avatar y estado vacío) se construyeron solo con primitivas del núcleo y se reutilizan en las doce pantallas.

4.6 Desarrollo de las interfaces

Se describe cada interfaz con sus capturas en el emulador Android. En cada figura el panel (a) es el mockup de la sección 2.4 y los siguientes muestran la pantalla real en distintos estados.

› I1 · Inicio de sesión institucional

Elementos. Pantalla de entrada con la marca sobre el color primario y una tarjeta blanca centrada. Contiene el logotipo de la EPIS, los campos de correo y contraseña con iconos, el botón para mostrar u ocultar la contraseña, el interruptor *Mantener la sesión iniciada*, el botón **Ingresar** y tres accesos de demostración por rol. Si el correo no termina en @unsa.edu.pe aparece el mensaje de error del panel (c).

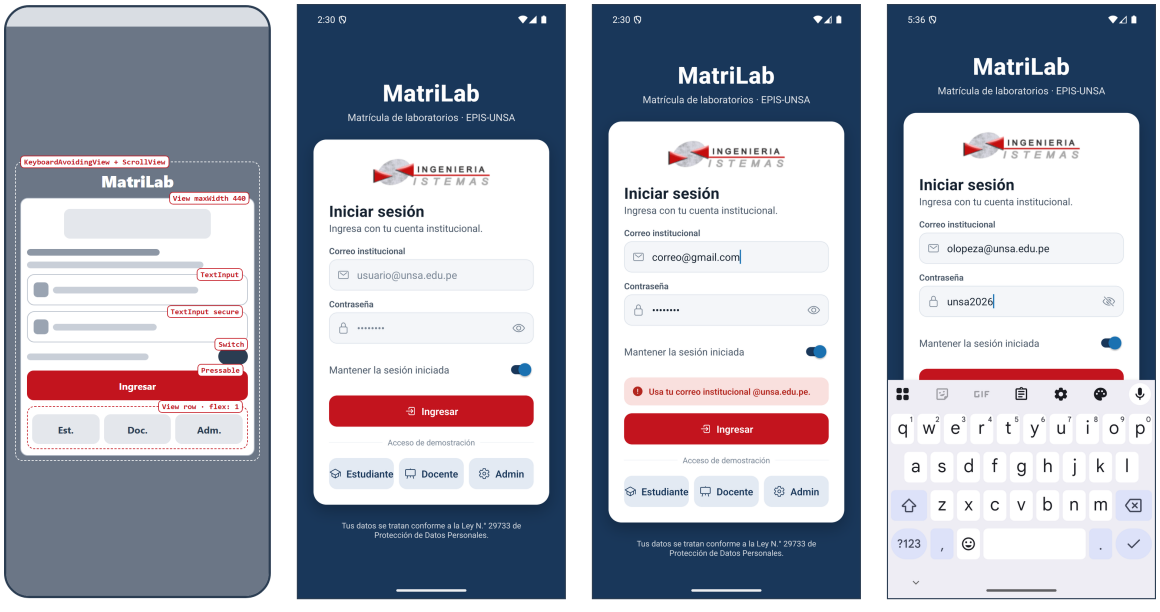
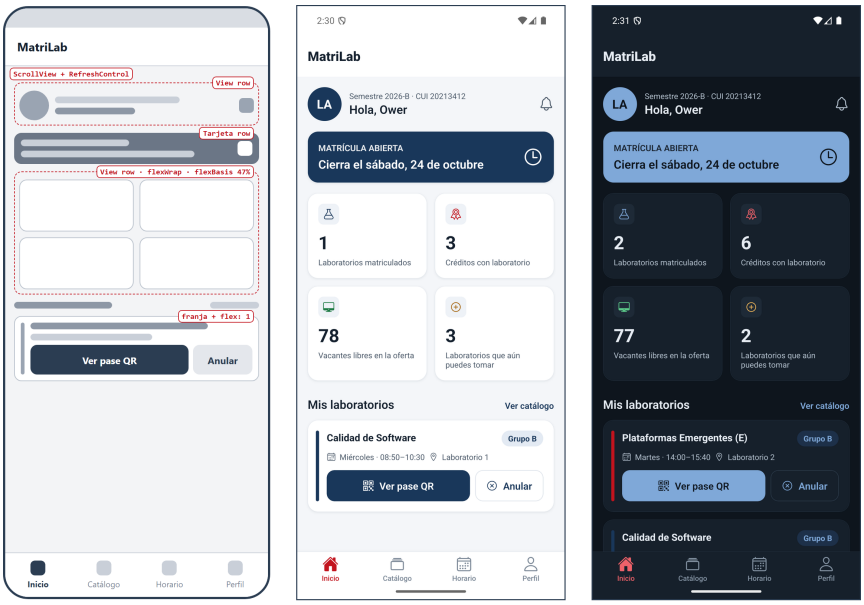


Figura 10. Inicio de sesión: (a) mockup, (b) pantalla inicial, (c) validación del dominio institucional, (d) formulario con el teclado abierto y la contraseña visible

Componentes y layout. La tarjeta está centrada verticalmente dentro de un contenedor desplazable y tiene un ancho máximo para no estirarse en tabletas ni en la web. Cuando aparece el teclado (panel d), KeyboardAvoidingView reduce el espacio disponible y el formulario sube, de modo que el campo activo sigue visible. Los tres accesos de demostración son una fila de botones que se reparten el ancho por igual. Como el fondo es oscuro, la barra de estado del sistema se pinta en claro.

I2 · Panel del estudiante

Elementos. Muestra el saludo con avatar e iniciales, la tarjeta con el estado de la ventana de matrícula y su fecha de cierre, cuatro indicadores (laboratorios matriculados, créditos, vacantes libres en la oferta y laboratorios que aún puede tomar) y la lista *Mis laboratorios*. Cada matrícula tiene una franja con el color del curso y los botones **Ver pase QR** y **Anular**.

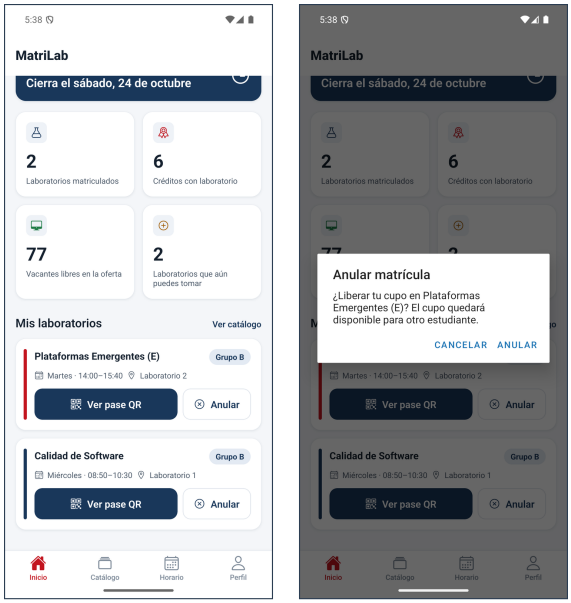


(a) Mockup (b) Tema claro (c) Tema oscuro

Figura 11. Panel del estudiante: (a) mockup, (b) tema claro con una matrícula, (c) tema oscuro después de matricularse en Plataformas Emergentes

Componentes y layout. Toda la pantalla se desplaza y admite el gesto de deslizar hacia abajo para actualizar. Los cuatro indicadores están en una fila que salta de línea: cada uno ocupa casi media fila, de modo que en un teléfono se forman dos columnas y en una pantalla ancha caben cuatro sin cambiar nada. La franja de color de cada matrícula es una vista angosta y el contenido ocupa el resto de la fila. Al comparar (b) y (c) se ve que los indicadores se actualizan solos: 2 laboratorios, 6 créditos y una vacante menos en la oferta.

Al pulsar **Anular** aparece el diálogo nativo de Android con la opción destructiva destacada. Si se confirma, el cupo vuelve a la oferta al instante y el semáforo del grupo se recalcula en el catálogo.

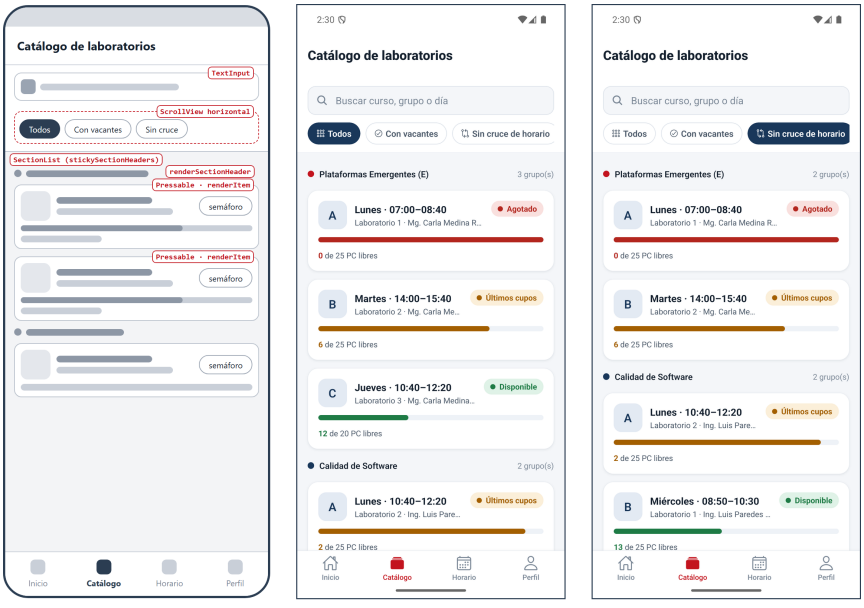


(a) Dos matrículas (b) Alert nativo

Figura 12. Panel del estudiante: (a) dos laboratorios matriculados, (b) diálogo nativo para anular una matrícula

I3 · Catálogo de grupos con semáforo

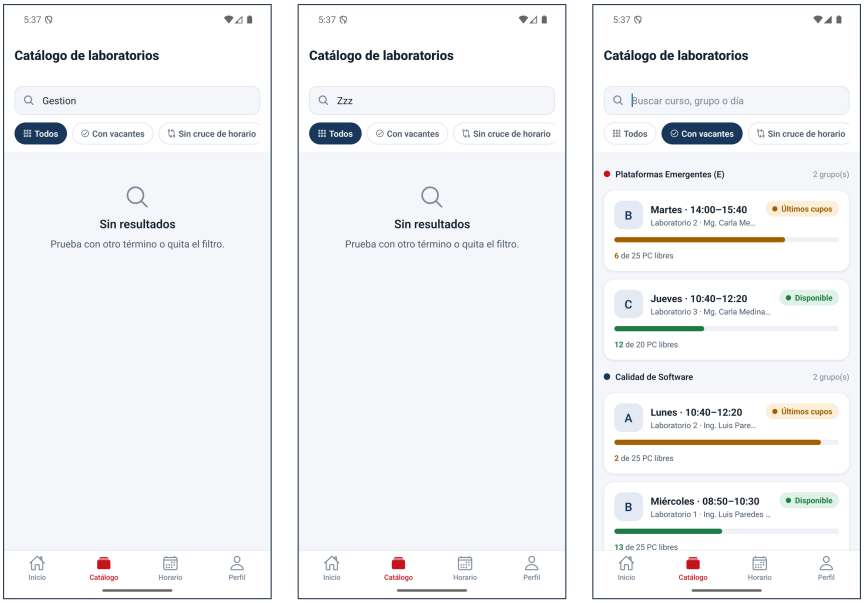
Elementos. Lista todos los grupos de laboratorio agrupados por curso. Cada tarjeta muestra la letra del grupo, día y hora, ambiente y docente, la insignia del semáforo (*Agotado*, *Últimos cupos*, *Disponible*), una barra de ocupación y el texto *N de M PC libres*. Con el filtro *Sin cruce de horario* (panel c) desaparece el grupo C de Plataformas Emergentes, que coincide con la teoría de Gestión de Proyectos del jueves.



(a) Mockup (b) Todos (c) Sin cruce

Figura 13. Catálogo: (a) mockup, (b) todos los grupos, (c) filtro Sin cruce de horario

Componentes y layout. La lista está dividida en secciones, una por curso, y el nombre del curso queda fijo arriba mientras se desplazan sus grupos. Los filtros son chips dentro de una fila con desplazamiento horizontal, de modo que pueden agregarse más sin romper el diseño. En cada tarjeta la letra del grupo es un cuadro de tamaño fijo, el texto ocupa el espacio libre y la insignia queda al final de la fila.



(a) Búsqueda (b) Sin resultados (c) Con vacantes

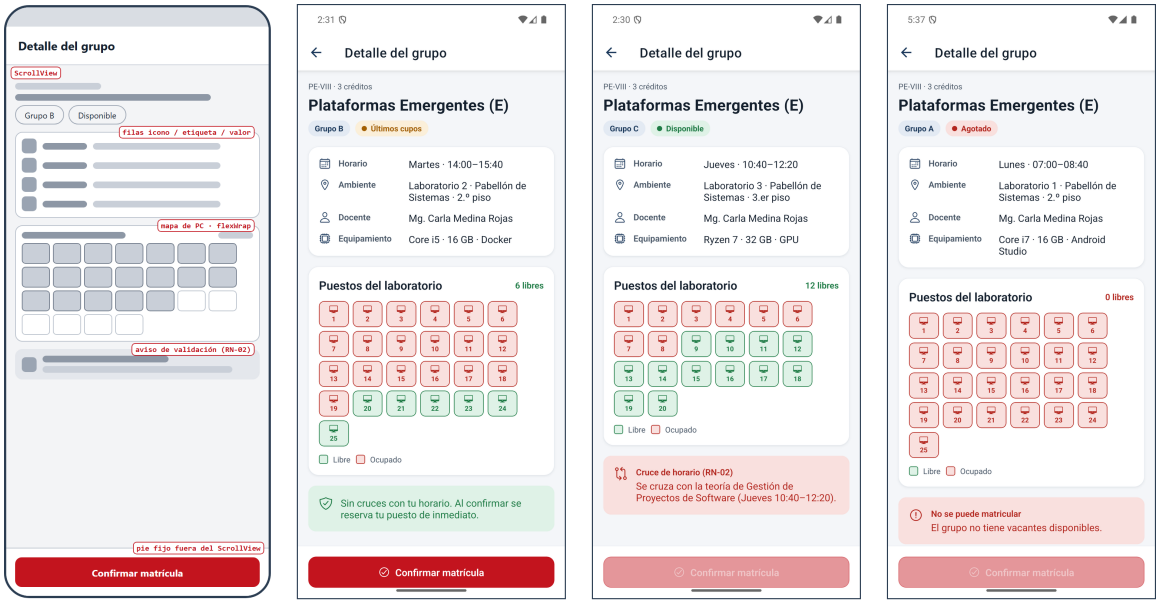
Figura 14. Catálogo: (a) búsqueda por nombre de curso, (b) estado vacío sin resultados, (c) filtro Con vacantes

El buscador filtra por curso, grupo o día mientras se escribe (a). Si no hay coincidencias aparece un estado vacío con un mensaje que orienta al usuario (b). El filtro *Con vacantes* oculta los grupos agotados, como el grupo A de Plataformas Emergentes (c).

› I4 · Detalle del grupo y confirmación

Elementos. Se abre como hoja modal desde el catálogo. Presenta el curso, las insignias de grupo y semáforo, una tarjeta con horario, ambiente, docente y equipamiento, y el **mapa de puestos**: una celda por computadora, en rojo las ocupadas y en verde las libres. Debajo, un aviso verde confirma que no hay cruces (b); si hay un cruce, el aviso rojo indica con qué curso y en qué horario (c); si el grupo está lleno, informa que no hay vacantes (d). En los dos últimos casos el botón **Confirmar matrícula** queda deshabilitado.

Componentes y layout. El mapa de puestos es una fila que salta de línea con celdas de tamaño fijo: con 20, 25 o 30 computadoras la cuadrícula se reacomoda sola en cualquier ancho. El botón de confirmación está en un pie fuera del área desplazable, siempre visible. Al confirmar se muestra un indicador de carga y luego el detalle se reemplaza por el pase, para que el botón Atrás no regrese a un formulario ya enviado.



(a) Mockup (b) Disponible (c) Cruce (d) Agotado

Figura 15. Detalle del grupo: (a) mockup, (b) grupo B disponible, (c) grupo C bloqueado por cruce con la teoría (RN-02), (d) grupo A agotado

› I5 · Pase de laboratorio con QR

Elementos. Es la constancia digital que el estudiante muestra en la puerta del laboratorio. La parte superior, en color primario, lleva el curso, el grupo, el horario y el sello *Confirmada*; la inferior, el código QR, el código de verificación de ocho caracteres y los datos del estudiante, el ambiente y la matrícula. **Compartir pase** abre la hoja nativa del sistema con el texto del pase listo para enviarse (c).

Componentes y layout. La tarjeta está dividida en dos zonas. La perforación entre ellas se dibuja con dos círculos del color del fondo colocados con posición absoluta en los bordes y una línea discontinua. El sello *Confirmada* también se ubica con posición absoluta en la esquina superior derecha. El tamaño del QR se calcula a partir del ancho de la pantalla y los datos se ordenan en dos columnas de 50 % cada una.

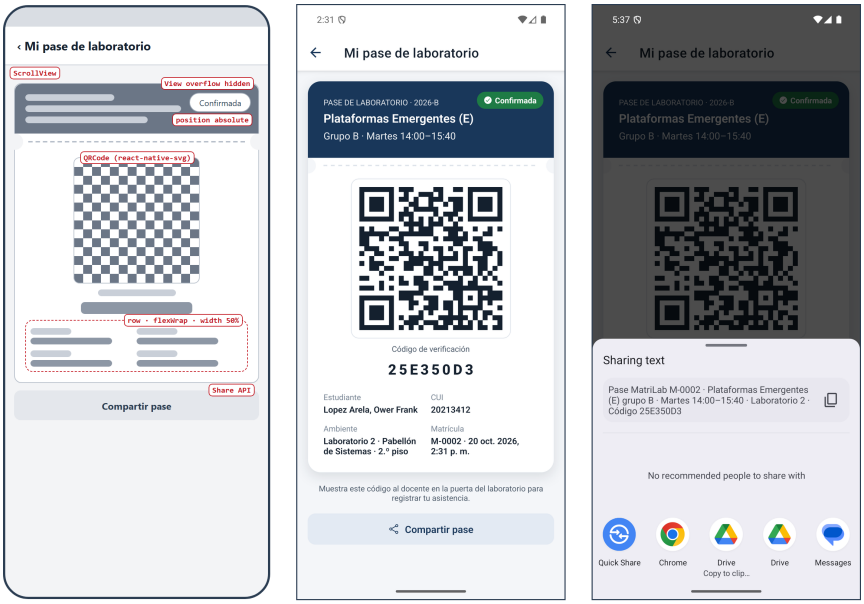
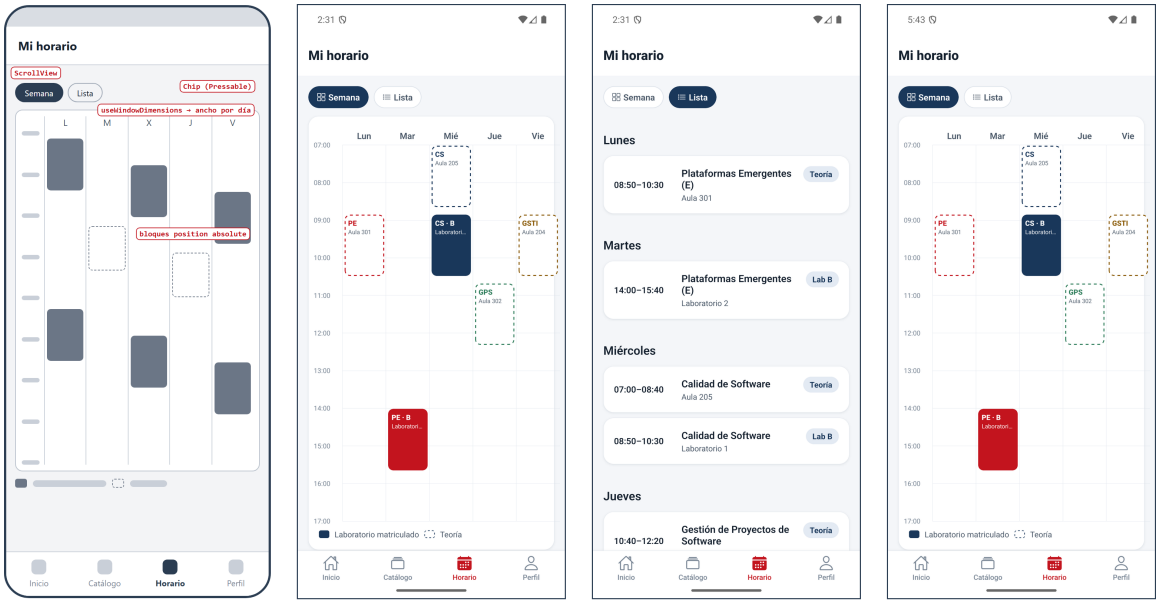


Figura 16. Pase de laboratorio: (a) mockup, (b) pase emitido tras confirmar la matrícula, (c) hoja nativa de compartir

› I6 · Horario semanal consolidado

Elementos. Consolida en una semana los laboratorios matriculados (bloques rellenos con el color del curso) y las clases de teoría (bloques con borde discontinuo). El selector *Semana / Lista* cambia a una vista de tarjetas agrupadas por día (c). Tras matricularse en Plataformas Emergentes aparece el nuevo bloque del martes de 14:00 a 15:40 (d).

Componentes y layout. Como React Native no tiene una cuadrícula al estilo de CSS Grid, el horario se construye a mano: una columna de horas y cinco columnas de día en fila, con el ancho de cada día calculado a partir del ancho de la ventana. Cada bloque se coloca con posición absoluta: su distancia desde arriba corresponde a la hora de inicio y su alto a la duración. Si la pantalla es muy angosta, la cuadrícula pasa a desplazarse en horizontal.

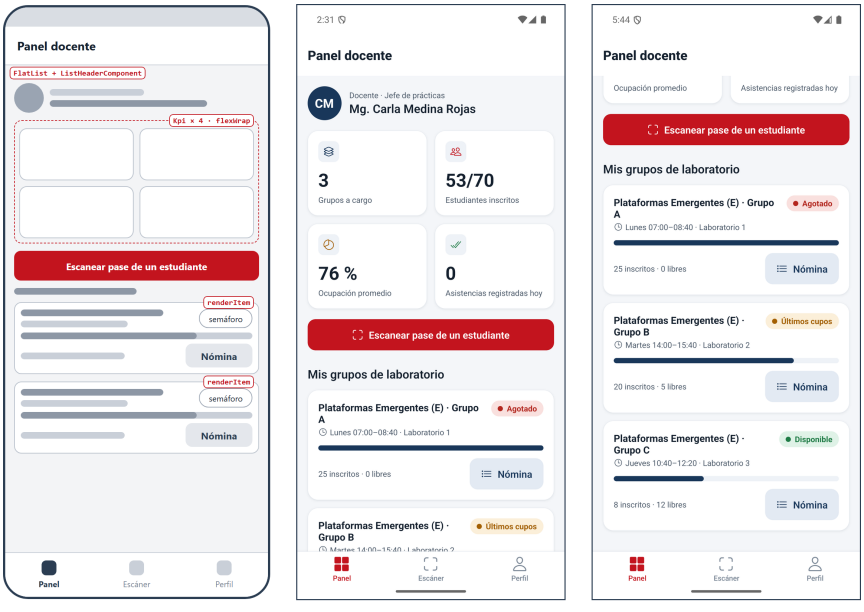


(a) Mockup (b) Semana (c) Lista (d) Con PE-B

Figura 17. Horario: (a) mockup, (b) vista semanal, (c) vista de lista por día, (d) semana después de matricularse en Plataformas Emergentes

I7 · Panel del docente

Elementos. Resume la carga del docente: grupos a cargo, estudiantes inscritos sobre la capacidad, ocupación promedio y asistencias registradas en el día. Un botón destacado lleva al escáner y debajo se listan los grupos con semáforo, barra de ocupación y acceso a la **Nómina** de cada uno.



(a) Mockup (b) Android (c) Asistencia

Figura 18. Panel del docente: (a) mockup, (b) panel con los grupos a cargo, (c) grupos después de registrar una asistencia

Componentes y layout. Toda la pantalla es una sola lista virtualizada: el saludo, los indicadores y el botón forman la cabecera de la propia lista y se desplazan junto con los grupos. Así se evita colocar una lista dentro de otro contenedor desplazable, situación que React Native desaconseja. Las cabeceras de cada tarjeta reparten el espacio entre el nombre del grupo y su insignia.

› I8 · Nómina y asistencia

Elementos. Lista a los estudiantes del grupo ordenados alfabéticamente, con número, nombre y CUI. Tocar una fila registra la asistencia con la hora y la fila pasa de *Pendiente* a una píldora verde. El resumen superior muestra presentes sobre el total, una barra de avance y un buscador por nombre o CUI. El pie ofrece **Exportar CSV** y **Escanear**.

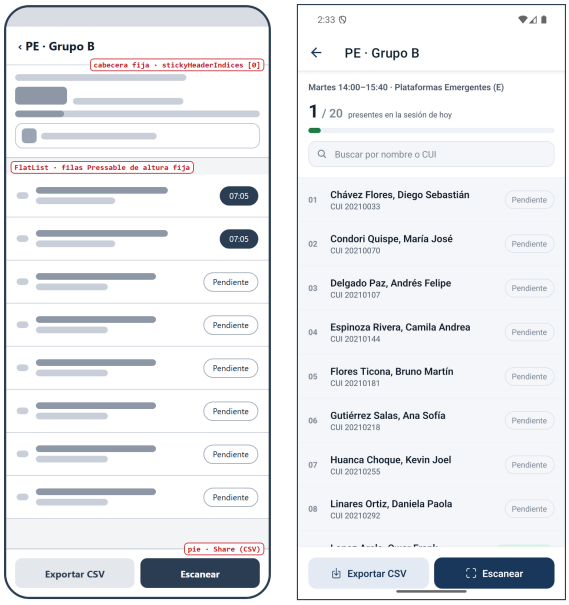
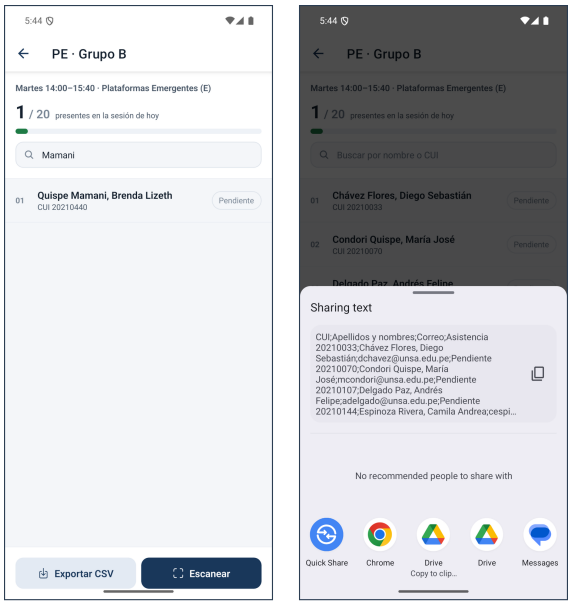


Figura 19. Nómina del grupo: (a) mockup, (b) dos asistencias registradas con su hora

Componentes y layout. Es una lista virtualizada cuyo primer elemento, el resumen, queda fijo arriba mientras se desplazan los estudiantes. Las filas tienen alto constante para que el desplazamiento sea uniforme, y cada una se anuncia al lector de pantalla como una casilla marcada o sin marcar. El título de la cabecera cambia según el curso y el grupo.

El buscador reduce la lista a medida que se escribe (a). **Exportar CSV** genera la nómina con separador punto y coma (el formato que Excel en español abre sin asistente) y la entrega a la hoja de compartir del sistema, desde donde puede enviarse por correo o guardarse en Drive (b).



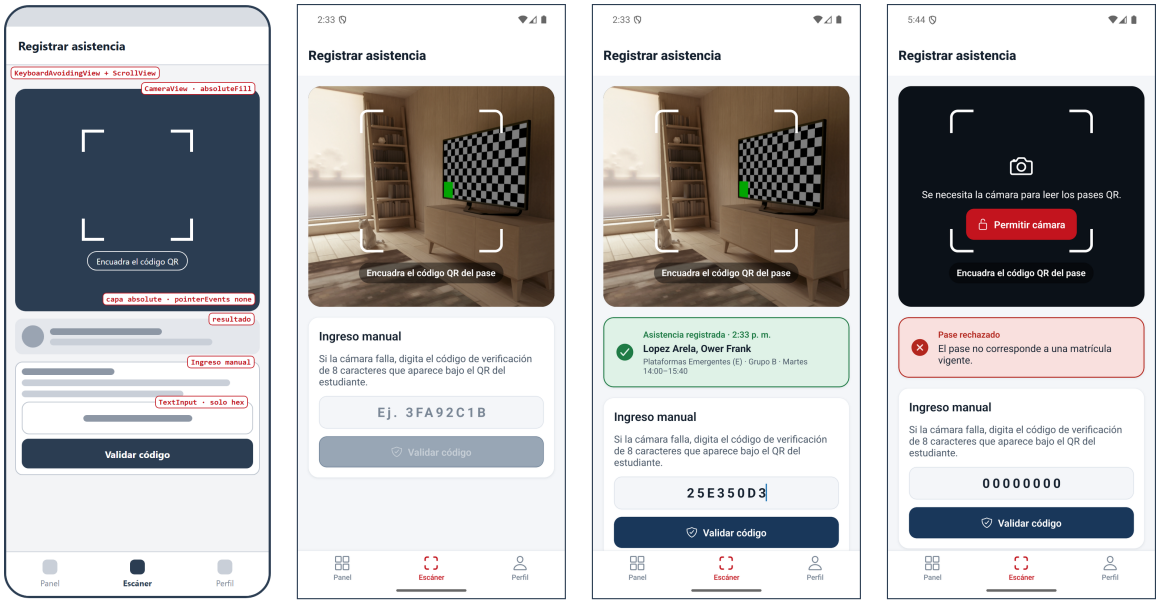
(a) Búsqueda (b) Exportar CSV

Figura 20. Nómina: (a) búsqueda por apellido, (b) exportación de la nómina en CSV con la hoja nativa de compartir

I9 · Escáner de asistencia

Elementos. Muestra la vista de la cámara con una guía de encuadre y lee el QR del pase. Si la cámara no está disponible, el docente puede digitar el código de verificación de ocho caracteres. El resultado aparece en una tarjeta verde con el nombre del estudiante, el curso y la hora (c), o en una roja si el pase no corresponde a una matrícula vigente o es de otro docente (d). En (d) se ve además el estado previo al permiso de cámara, con el botón *Permitir cámara*.

Componentes y layout. La vista de la cámara cubre todo su recuadro y encima se coloca otra capa transparente a los toques, que dibuja las cuatro esquinas del marco y la leyenda. Cada esquina es una vista con posición absoluta y dos bordes. El campo manual convierte lo escrito a mayúsculas, solo acepta caracteres hexadecimales y limita la longitud a ocho, y el lector se bloquea unos segundos después de cada código para no registrar dos veces al mismo estudiante.



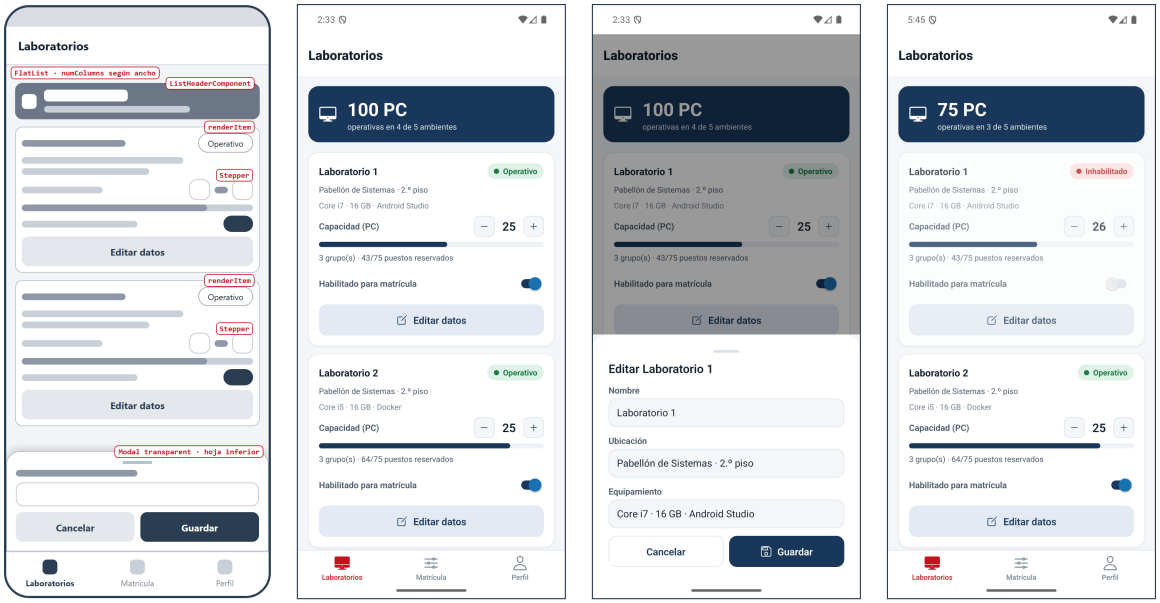
(a) Mockup (b) Cámara (c) Validado (d) Rechazado

Figura 21. Escáner: (a) mockup, (b) cámara activa en el emulador, (c) pase validado, (d) código rechazado

I10 · Gestión de laboratorios

Elementos. La coordinación ve el total de computadoras operativas y una tarjeta por ambiente con ubicación, equipamiento, capacidad ajustable con los botones − y +, barra de puestos reservados, interruptor de habilitación y botón **Editar datos**, que abre una hoja inferior para cambiar nombre, ubicación y equipamiento (c). Al desactivar un laboratorio su insignia pasa a *Inhabilitado* y el total de la cabecera baja de 100 a 75 PC (d).

Componentes y layout. La lista calcula el número de columnas según el ancho: una por debajo de 600 dp, dos hasta 960 dp y tres en adelante (ver sección 4.7). La edición usa el Modal nativo con fondo transparente: una capa semitransparente ocupa la pantalla y al tocarla se cierra, mientras la hoja con esquinas superiores redondeadas queda abajo.

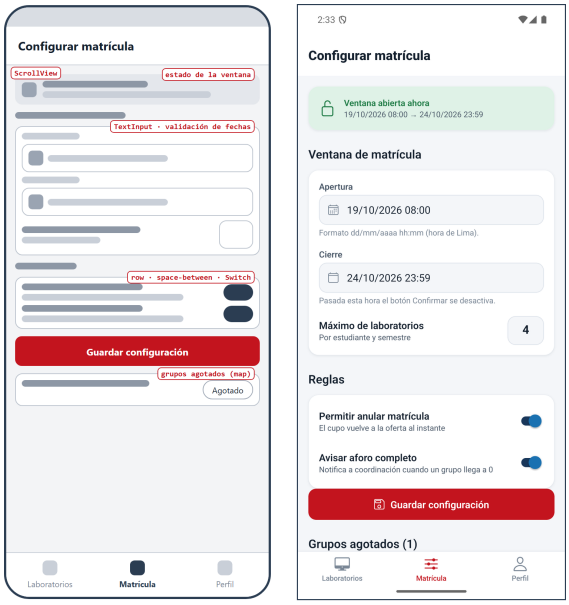


(a) Mockup (b) Lista (c) Edición (d) Inhabilitado

Figura 22. Gestión de laboratorios: (a) mockup, (b) lista de ambientes, (c) edición en la hoja modal, (d) laboratorio inhabilitado y capacidad ajustada

I11 · Configuración de la matrícula

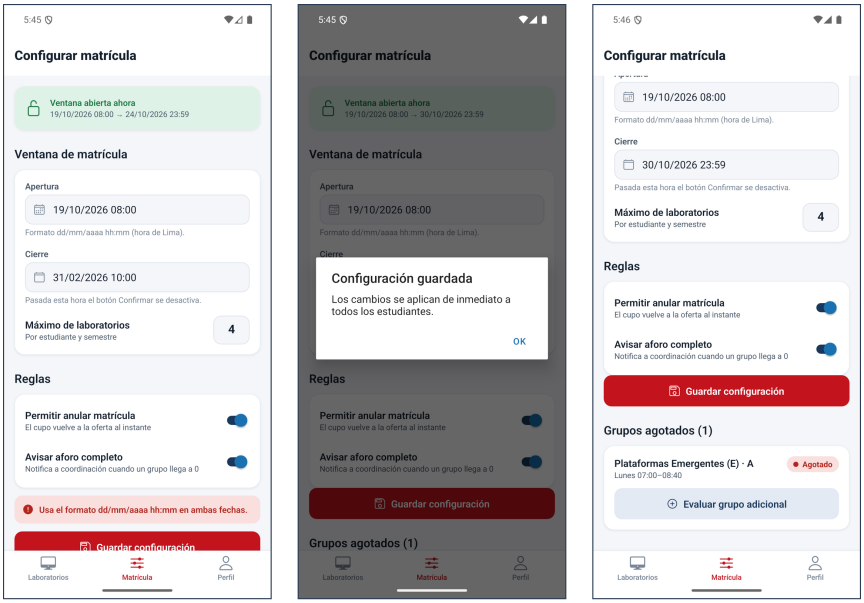
Elementos. Permite fijar la apertura y el cierre de la ventana de matrícula, el máximo de laboratorios por estudiante y dos reglas: permitir anular matrículas y avisar cuando un grupo se agota. Muestra el estado actual de la ventana y, al final, la lista de grupos agotados con la opción de solicitar un grupo adicional.



(a) Mockup (b) Android

Figura 23. Configuración de la matrícula: (a) mockup, (b) ventana abierta y reglas

Componentes y layout. Es un formulario desplazable en el que tocar un botón no solo cierra el teclado. Cada regla es una fila con el texto a la izquierda y el interruptor a la derecha. Antes de guardar se valida el formato de fecha, que el cierre sea posterior a la apertura y que el máximo esté entre 1 y 6.



(a) Fecha inválida (b) Guardado (c) Agotados

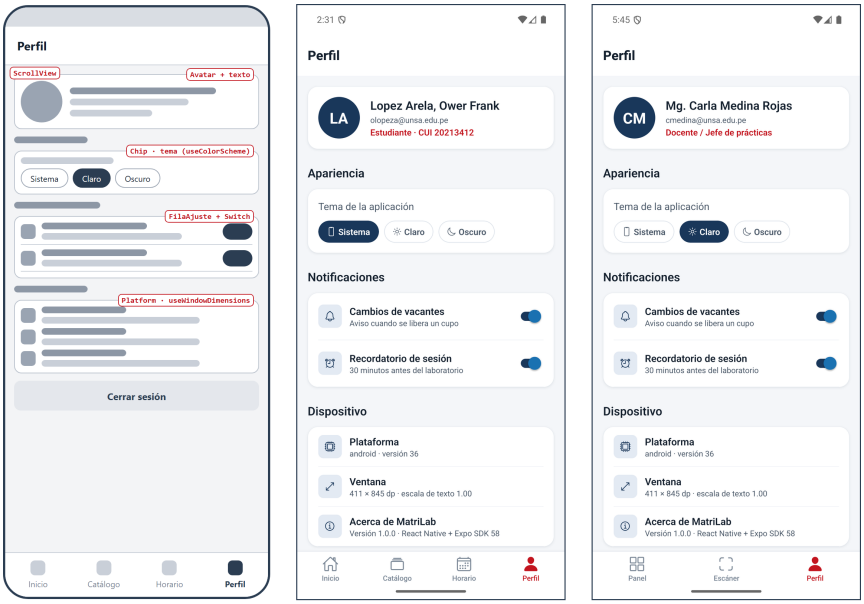
Figura 24. Configuración: (a) fecha inválida rechazada, (b) confirmación nativa al guardar, (c) grupos agotados al final del formulario

Una fecha inexistente como el 31 de febrero se rechaza con un mensaje sobre el botón (a). Con datos válidos aparece el diálogo nativo *Configuración guardada* y la ventana se actualiza para todos los estudiantes (b). Al final del formulario se listan los grupos sin vacantes con la opción *Evaluar grupo adicional* (c).

› I12 · Perfil y ajustes

Elementos. Es común a los tres roles. Muestra los datos del usuario y su rol, el selector de tema (*Sistema*, *Claro*, *Oscuro*), dos interruptores de notificaciones, información del dispositivo y el botón **Cerrar sesión**, que reinicia la navegación hacia el inicio de sesión.

Componentes y layout. Las filas de ajustes repiten la misma estructura: icono, texto que ocupa el espacio libre y un control a la derecha. La sección *Dispositivo* evidencia el entorno de ejecución: en el emulador indica *android · versión 36* y el tamaño de la ventana en dp. El tema elegido se aplica de inmediato a toda la aplicación.



(a) Mockup (b) Estudiante (c) Docente

Figura 25. Perfil y ajustes: (a) mockup, (b) perfil del estudiante, (c) perfil del docente

4.7 Adaptación a la orientación y a otras plataformas

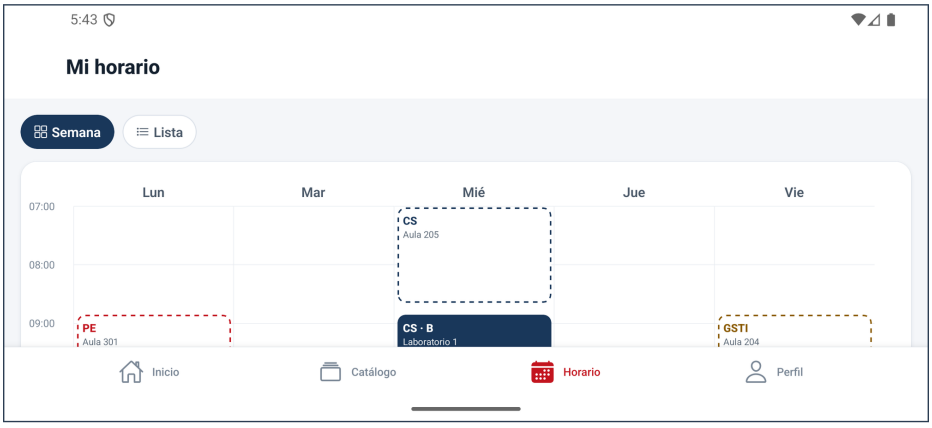
Para comprobar que el layout responde al tamaño de la ventana se giró el emulador a horizontal. La ventana pasa a unos 915 dp de ancho y, sin código específico para la rotación, la lista de laboratorios pasa a dos columnas y el horario reparte el ancho adicional entre los cinco días (Figura 26).

El mismo código se exportó a la web. react-native-web traduce los componentes a elementos del navegador y la aplicación funciona igual (Figura 27); la versión publicada está en `https://owerlopez.github.io/matrilab-epis-react-native/`.

En una pantalla ancha la lista de laboratorios llega a tres columnas y en un ancho intermedio a dos, mientras la tarjeta del inicio de sesión mantiene su ancho máximo centrada (Figura 28).

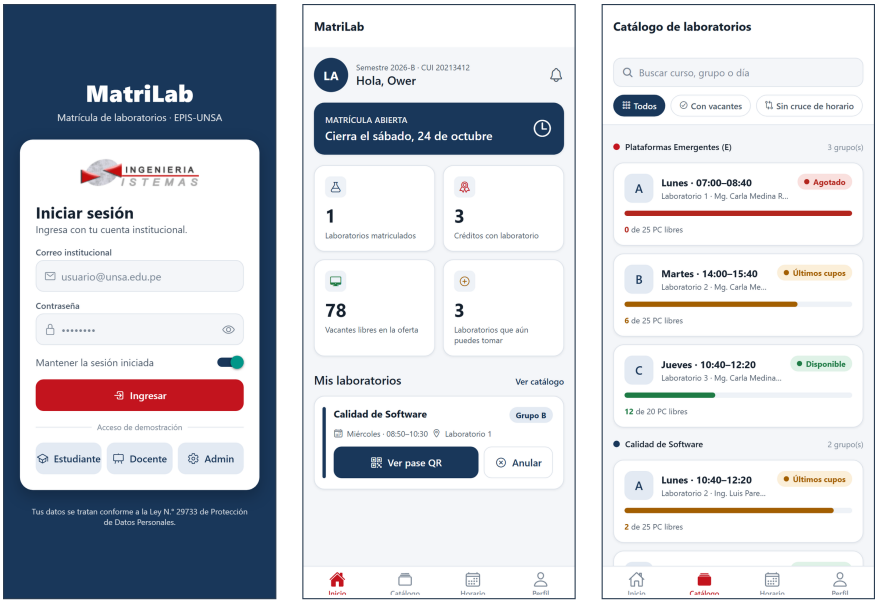


(a) Laboratorios en dos columnas



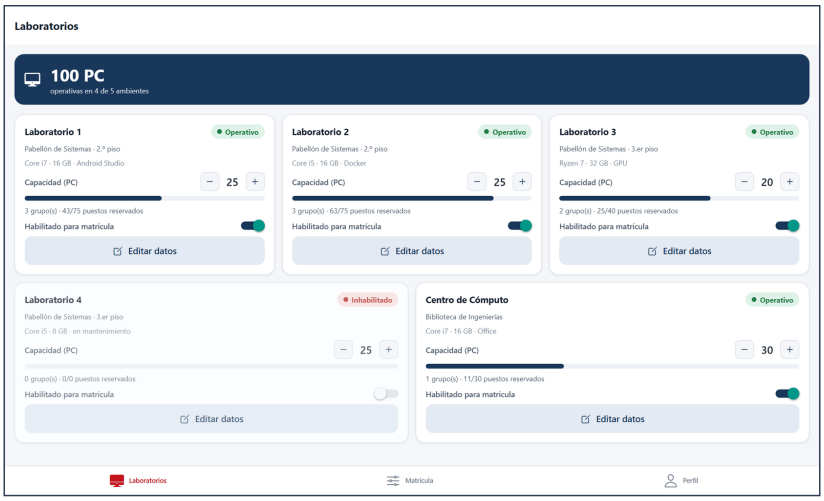
(b) Horario semanal

Figura 26. Orientación horizontal en el emulador: (a) laboratorios en dos columnas, (b) horario semanal

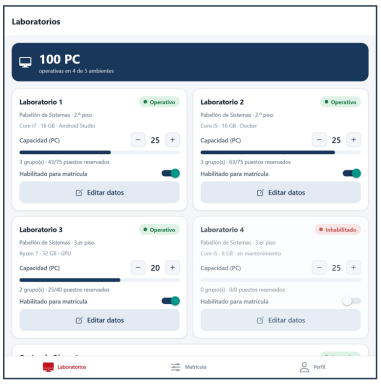


(a) Inicio de sesión (b) Panel (c) Catálogo

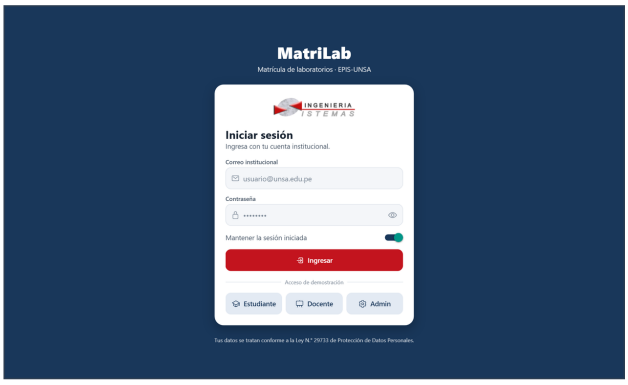
Figura 27. La misma aplicación en un navegador web: inicio de sesión, panel del estudiante y catálogo



(a) 1366 px: tres columnas



(b) 820 px: dos columnas



(c) Inicio de sesión

Figura 28. Versión web en distintos anchos: (a) 1366 px con tres columnas, (b) 820 px con dos columnas, (c) inicio de sesión centrado

4.8 Pruebas automatizadas y verificación

Las reglas de negocio y el estado se verificaron con **31 pruebas unitarias** en Jest [22], [23], que se ejecutan en menos de cinco segundos porque no necesitan emulador. La verificación de tipos de TypeScript en modo estricto también termina sin errores (Figura 29). El dominio alcanza el 100 % de líneas cubiertas.

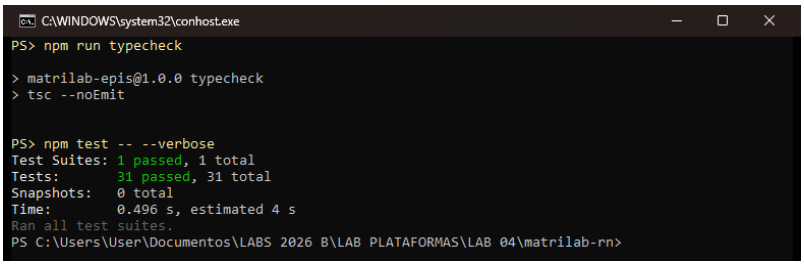


Figura 29. Verificación de tipos sin errores y 31 pruebas aprobadas en la consola

Tabla 11. Pruebas unitarias agrupadas por regla verificada

Grupo	Pruebas	Qué se verifica
Conversión de horas	1	07:00 equivale a 420 minutos y 640 minutos a 10:40.
RN-02 cruce de horario	3	Solape el mismo día; bloques contiguos no se cruzan; días distintos no se cruzan.
Semáforo	1	Rojo sin cupo, ámbar con menos del 30 % libre y verde en otro caso.
Validación de matrícula	8	Acepta un grupo libre; rechaza cruce con teoría y con otro laboratorio, mismo grupo, mismo curso, grupo agotado, ventana cerrada y límite.
Reserva atómica (RF-05)	4	Suma un cupo sin alterar el estado anterior; no reserva sin cupo; nunca supera el aforo; liberar no deja negativos.
Pase QR	3	El contenido generado se verifica; un QR alterado se rechaza; la huella es estable y de 8 caracteres.
Horario y nómina	4	Orden de bloques; ventana abierta; nómina con tantas filas como cupos; CSV con punto y coma.
Estado global	7	Matricular y anular actualizan cupos; sin sesión no se matricula; acciones de administración; la asistencia se marca una sola vez.

Además de las pruebas unitarias, todas las capturas de este informe se obtuvieron con un recorrido automatizado sobre el emulador que localiza cada control por su etiqueta de accesibilidad, lo toca o escribe en él y guarda la pantalla. Ese recorrido funciona como prueba de la navegación completa: iniciar sesión con cada rol, matricularse, ver y compartir el pase, marcar asistencia, validar y rechazar códigos, editar laboratorios, guardar la configuración y girar la pantalla.

4.9 Registro de cambios

La guía pide dejar constancia de las modificaciones realizadas durante el trabajo. La Tabla 12 registra los cambios que surgieron al probar la aplicación en el emulador.

Tabla 12. Cambios realizados durante el desarrollo y su motivo

N.°	Pantalla o archivo	Cambio	Motivo
1	Inicio de sesión	Barra de estado en color claro.	La hora y los iconos del sistema quedaban oscuros sobre el fondo azul.
2	Laboratorios	Una columna por debajo de 600 dp (antes dos desde 400 dp).	En el Pixel 7a (411 dp) las tarjetas quedaban comprimidas.
3	Configuración de Expo	Orientación libre en lugar de solo vertical.	El sistema bloqueaba el giro y no se podía comprobar el layout horizontal.
4	Escáner	El campo manual solo acepta hexadecimal en mayúsculas y 8 caracteres.	Evitar rechazos por minúsculas o espacios al dictar el código.
5	Nómina	Se quitó el cálculo fijo de la posición de cada fila.	La cabecera fija tiene alto variable y desplazaba las filas.
6	Configuración de TypeScript	Se declararon los tipos de Jest y Node.	La verificación de tipos fallaba en los archivos de prueba.
7	Pruebas	Se agregaron 8 pruebas (de 23 a 31).	Cubrir el grupo ya matriculado, el cruce entre laboratorios y las acciones del estado.
8	Versión web	Ruta base para publicar en GitHub Pages.	Servir la aplicación desde una subcarpeta del sitio.

4.10 Repositorio y despliegue

Todo el trabajo está en el repositorio público `https://github.com/OwerLopez/matrilab-epis-react-native` (Figura 30): el código de la aplicación, las pruebas, las capturas, el script del recorrido automatizado y este informe con sus fuentes LaTeX y diagramas.

En cada cambio publicado en la rama principal, GitHub Actions ejecuta la verificación de tipos y las 31 pruebas, exporta la versión web y la publica en GitHub Pages (Figura 31). Si una prueba falla, el despliegue no se realiza.

Para probar la aplicación basta abrir la versión web o, con el proyecto clonado, iniciar Metro y abrirlo en Expo Go. Cuentas de demostración: olopeza@unsa.edu.pe, cmedina@unsa.edu.pe o laboratorios.epis@unsa.edu.pe con la contraseña unsa2026, o los botones *Estudiante*, *Docente* y *Admin* del inicio de sesión.

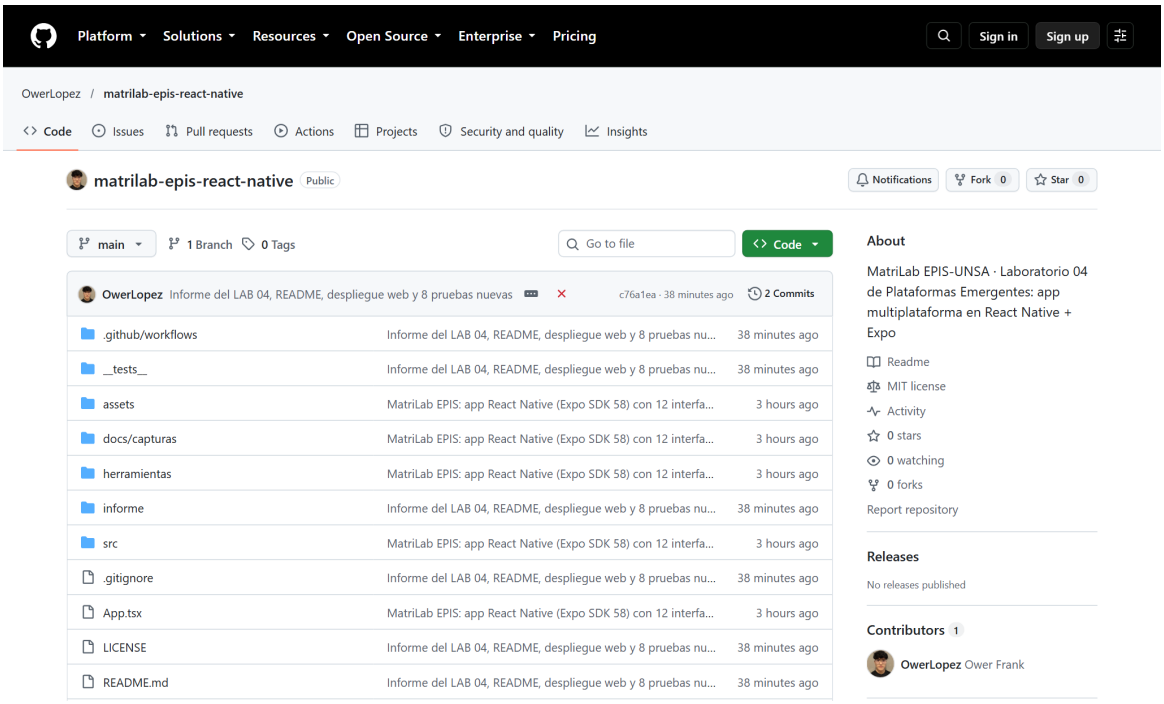


Figura 30. Repositorio del proyecto en GitHub

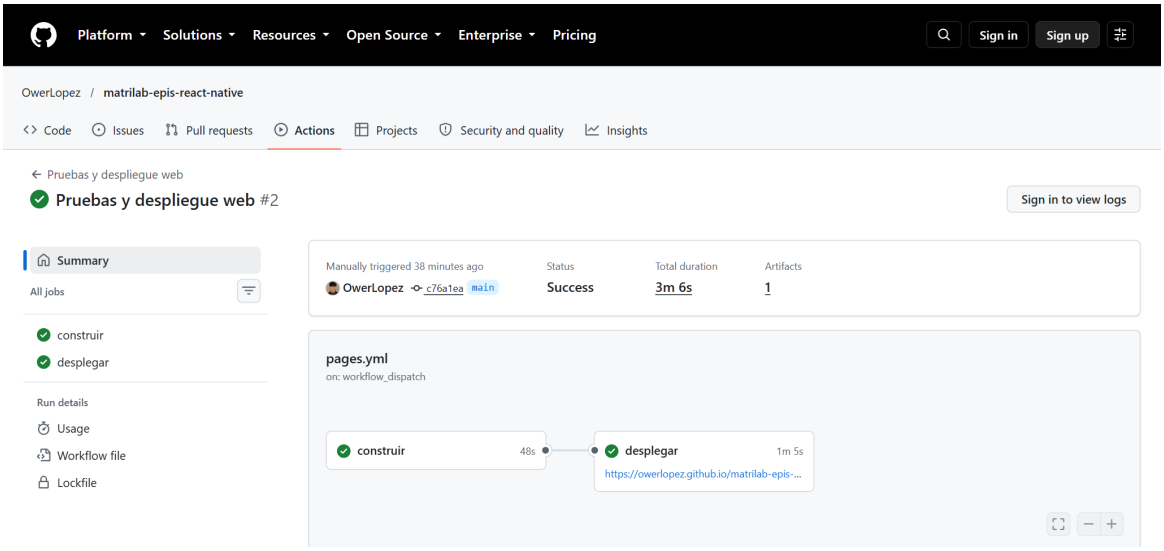


Figura 31. Flujo de GitHub Actions: trabajos construir y desplegar completados

5. APRECIACIÓN DE LA HERRAMIENTA

La valoración se basa en lo que el equipo observó al construir y probar MatriLab con React Native y Expo.

5.1 Características favorables

A. Interfaces nativas reales a partir de un solo código

Cada componente del núcleo se convierte en una vista nativa del sistema [2]. En las capturas se ve que el interruptor de la configuración es el de Android, el teclado y la hoja de compartir son los del teléfono, los diálogos de anular y guardar son los del sistema y la pila de navegación usa las transiciones y el gesto de retroceso nativos. Con ese mismo código, sin cambios, la aplicación se exportó a la web (Figura 27). Las diferencias por plataforma que sí hubo que escribir fueron mínimas: el comportamiento del teclado y el efecto ripple de Android. Para un proyecto que debe llegar a estudiantes con Android y con iPhone, esto reduce a la mitad el trabajo frente al desarrollo nativo separado.

B. Ciclo de desarrollo rápido y ecosistema JavaScript

Con Expo Go no fue necesario compilar proyectos de Android Studio: el emulador cargaba la aplicación desde Metro (Figura 7) y *Fast Refresh* aplicaba cada cambio de estilo en uno o dos segundos conservando el estado de la pantalla, lo que permitió ajustar márgenes, columnas y colores viendo el resultado al instante. El ecosistema aportó piezas maduras: TypeScript detectó errores de navegación entre pantallas antes de ejecutarlas, una librería resolvió el código QR y Jest probó las reglas de negocio en segundos. El mismo lenguaje sirvió para la aplicación, las pruebas y el recorrido automatizado de capturas.

5.2 Características desfavorables

A. Dependencia de la compatibilidad entre versiones

Una aplicación React Native depende de una cadena de versiones que deben coincidir: Expo SDK, React Native, React, cada librería con código nativo y la versión de Expo Go instalada en el teléfono, que solo abre proyectos de su mismo SDK. Fue obligatorio instalar las dependencias con el instalador de Expo para no romper esa cadena. El ritmo de cambios también se notó: el SafeAreaView del núcleo figura como obsoleto [16], la Nueva Arquitectura cambió el comportamiento interno de varias librerías y TypeScript 6 dejó de incluir tipos globales por defecto, lo que obligó a corregir la configuración (cambio 6). En un proyecto que dure varios semestres, actualizar dependencias requiere tiempo dedicado.

B. Layout limitado a Flexbox y diferencias entre plataformas

React Native solo ofrece Flexbox y posición absoluta: no hay CSS Grid ni *media queries*. La cuadrícula del horario tuvo que calcularse a mano con las dimensiones de la ventana y posición absoluta, y la rejilla adaptable de laboratorios obligó a volver a crear la lista al cambiar de ancho. Algunos componentes se comportan distinto en cada sistema (teclado, sombras, ripple) y hay que probar en ambos. Además, lo que ocurre del lado nativo exige conocer las herramientas de Android: los permisos de cámara de Expo Go, el teclado del emulador y la rotación tuvieron que resolverse desde la línea de comandos del emulador.

6. CONCLUSIONES

- Se desarrolló MatriLab EPIS con React Native 0.88 y Expo SDK 58: doce interfaces para tres roles que se ejecutan como aplicación nativa en Android y como aplicación web con el mismo código, usando solo componentes del núcleo del framework para organizar la interfaz.
- El árbol de problemas y el árbol de objetivos justificaron cada pantalla: los cinco medios fundamentales se tradujeron en trece funcionalidades y estas en las doce interfaces (Tabla 3).
- La investigación de componentes mostró que el layout en React Native descansa en Flexbox con valores por defecto distintos a los de la web, en listas virtualizadas para colecciones grandes y en posición absoluta y dimensiones de ventana para superposiciones y adaptación. La rejilla de laboratorios pasó de una a dos y tres columnas según el ancho sin código adicional.
- Las capturas de los distintos estados (errores de validación, cruces, grupos agotados, diálogos y hojas nativas, tema oscuro y orientación horizontal) confirman que cada regla de negocio tiene una respuesta visible en la interfaz.
- Frente a FlutterFlow (LAB 02), React Native exige escribir más, pero da control total sobre el layout y el estado y produce interfaces con los controles nativos de cada sistema; el costo principal es mantener la compatibilidad de versiones del ecosistema.

7. ANEXOS

El código fuente completo de la aplicación está disponible en el repositorio `https://github.com/OwerLopez/matrilab-epis-react-native`. En este anexo se reúnen capturas complementarias.

Anexo A. Tema oscuro

Al elegir el tema oscuro desde el perfil, toda la aplicación cambia de paleta: fondos, tarjetas, textos y los colores del semáforo usan sus variantes oscuras manteniendo el contraste (Figura 32). En el detalle (c) se ve otra regla de negocio: el estudiante ya tiene un grupo de Plataformas Emergentes y no puede tomar un segundo grupo del mismo curso.

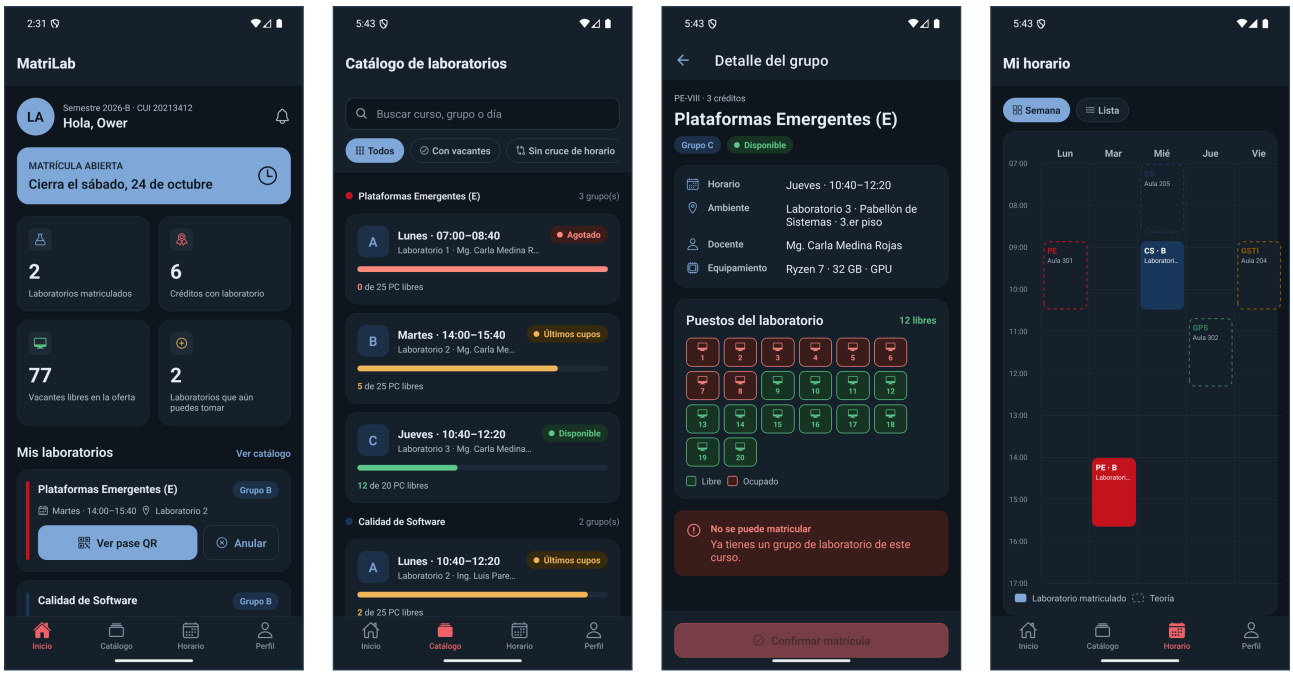


Figura 32. Tema oscuro: (a) panel del estudiante, (b) catálogo, (c) detalle con la regla de un grupo por curso, (d) horario

Anexo B. Desarrollo con Expo Go

El menú de desarrollo de Expo Go (Figura 8, panel b) permite recargar la aplicación, activar el monitor de rendimiento, inspeccionar elementos de la interfaz y abrir las herramientas de depuración. Se usó sobre todo el inspector de elementos para revisar los márgenes y los contenedores de cada pantalla.

8. REFERENCIAS

[1] Meta Platforms, Inc., *Layout with Flexbox*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/flexbox

[2] Meta Platforms, Inc., *Core Components and Native Components*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/intro-react-native-components

[3] Meta Platforms, Inc., *ScrollView*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/scrollview

[4] Meta Platforms, Inc., *FlatList*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/flatlist

[5] Meta Platforms, Inc., *SectionList*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/sectionlist

[6] Meta Platforms, Inc., *StyleSheet*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/stylesheet

[7] Meta Platforms, Inc., *Pressable*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/pressable

[8] Meta Platforms, Inc., *KeyboardAvoidingView*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/keyboardavoidingview

[9] Meta Platforms, Inc., *Modal*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/modal

[10] Meta Platforms, Inc., *useWindowDimensions*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/usewindowdimensions

[11] Meta Platforms, Inc., *Platform*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/platform

[12] Meta Platforms, Inc., *Height and Width*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/height-and-width

[13] Meta Platforms, Inc., *About the New Architecture*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/architecture/landing-page

[14] Expo, *Introduction to Expo*, Expo Documentation, 2026. [En línea]. Disponible en: https://docs.expo.dev/get-started/introduction/

[15] Meta Platforms, Inc., *Yoga: A portable layout engine*, 2026. [En línea]. Disponible en: https://www.yogalayout.dev/

[16] Meta Platforms, Inc., *SafeAreaView (deprecated)*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/safeareaview

[17] E. Ortegón, J. F. Pacheco y A. Prieto, *Metodología del marco lógico para la planificación, el seguimiento y la evaluación de proyectos y programas*, Serie Manuales n.° 42. Santiago de Chile: CEPAL, 2005.

[18] Meta Platforms, Inc., *Optimizing FlatList Configuration*, React Native Documentation, 2026. [En línea]. Disponible en: https://reactnative.dev/docs/optimizing-flatlist-configuration

[19] Expo, *Camera (expo-camera)*, Expo SDK Reference, 2026. [En línea]. Disponible en: https://docs.expo.dev/versions/latest/sdk/camera/

[20] React Navigation, *Native Stack Navigator*, React Navigation 7 Documentation, 2026. [En línea]. Disponible en: https://reactnavigation.org/docs/native-stack-navigator

[21] React Navigation, *Bottom Tabs Navigator*, React Navigation 7 Documentation, 2026. [En línea]. Disponible en: https://reactnavigation.org/docs/bottom-tab-navigator

[22] Expo, *Unit testing with Jest*, Expo Documentation, 2026. [En línea]. Disponible en: https://docs.expo.dev/develop/unit-testing/

[23] OpenJS Foundation, *Getting Started*, Jest Documentation, 2026. [En línea]. Disponible en: https://jestjs.io/docs/getting-started

[24] T. Atkins Jr., E. J. Etemad, R. Atanassov y O. Oyekunle (eds.), *CSS Flexible Box Layout Module Level 1*, W3C Candidate Recommendation, 2018. [En línea]. Disponible en: https://www.w3.org/TR/css-flexbox-1/

[25] International Organization for Standardization, *Ergonomics of human-system interaction — Part 210: Human-centred design for interactive systems*, ISO 9241-210:2019, 2019.

[26] B. Eisenman, *Learning React Native: Building Native Mobile Apps with JavaScript*, 2.ª ed. Sebastopol, CA, EE. UU.: O'Reilly Media, 2017.